第10章 分布式数据库系统与并行数据库系统

西安交通大学计算机科学与技术学院　　任雪斌

xuebinren@mail.xjtu.edu.cn

目录

10.1 概述

10.2 分布式数据存储

10.3 分布式数据库系统体系结构

10.4 分布式数据库管理系统

10.5 分布式查询处理和优化

10.6 分布式事务处理

10.7 联邦数据库和多数据库系统

10.8 并行数据库

10.1 概述

➢引言

- 随着传统的数据库技术日趋成熟、计算机网络技术的飞速发展和应用范围的扩大，以分布式为主要特征的数据库系统的研究与开发受到人们的注意。
- 分布式数据库是数据库技术与网络技术相结合的产物，在数据库领域已形成一个分支。

10.1 概述

➢分布式数据库的由来与发展

- 分布式数据库的研究始于20世纪70年代中期
- 世界上第一个分布式数据库系统SDD-1是由美国计算机公司(CCA)于1979年在DEC计算机上实现。
- 20世纪90年代以来，分布式数据库系统进入商品化应用阶段，传统的关系数据库产品均发展成以计算机网络及多任务操作系统为核心的分布式数据库产品，同时分布式数据库逐步向客户机 / 服务器模式发展。

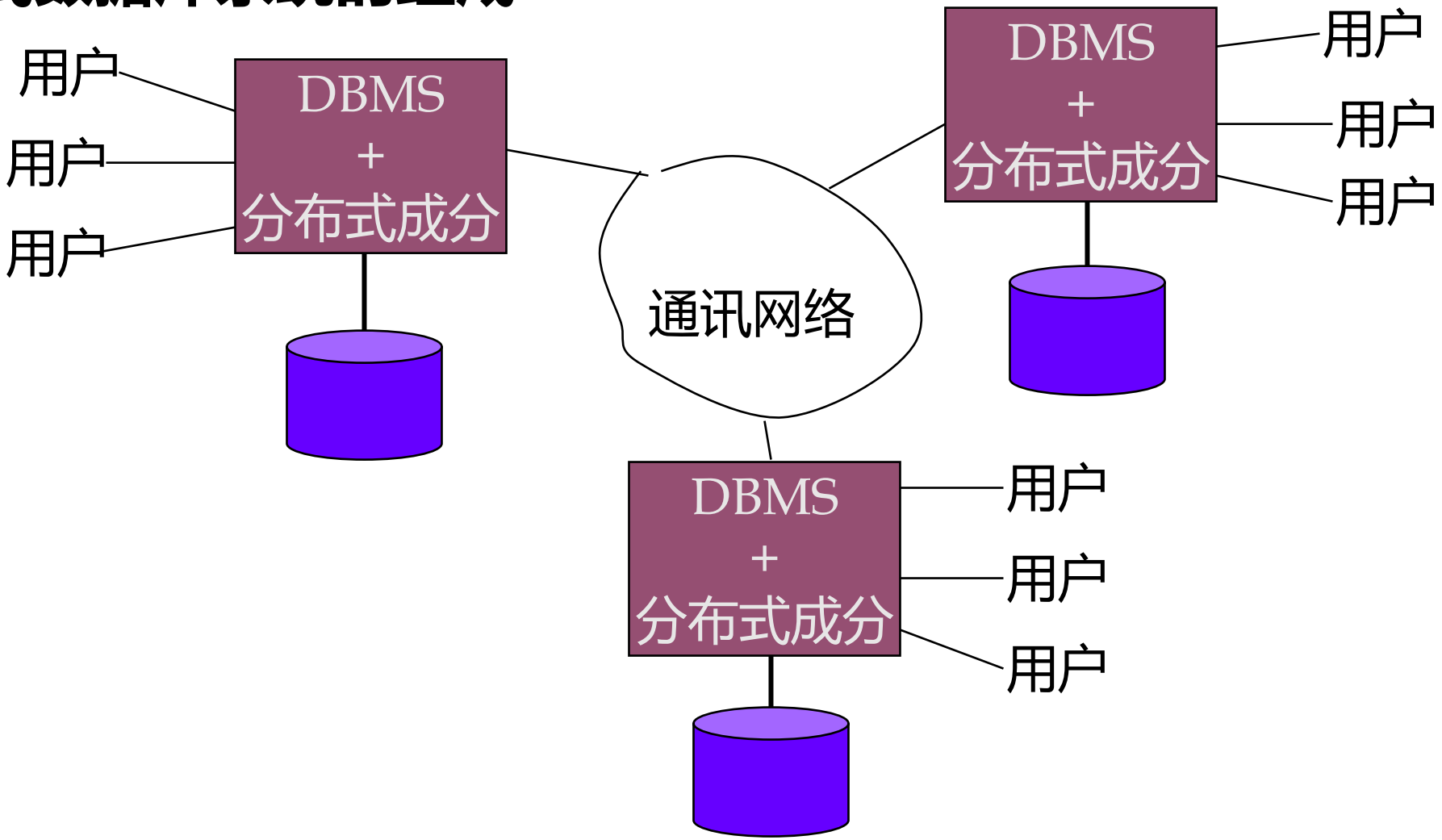
10.1 概述

➢分布式数据库系统的定义

- 分布式数据库系统(DDBS)是物理上分散，而逻辑上集中的数据库系统，系统中的数据分布存放在计算机网络的不同场地的计算机中，每一场地都有自治处理(即独立处理)能力并能完成局部应用，而每一场地也参与(至少一种)全局应用，程序通过网络通信子系统执行全局应用。
- 分布式数据库(DDB)是计算机网络环境中各场地上数据库的逻辑集合。
- 分布式数据库管理系统(DDBMS)是DDBS中的一组软件，负责管理分布环境下逻辑集成数据的存取、一致性和完备性 。

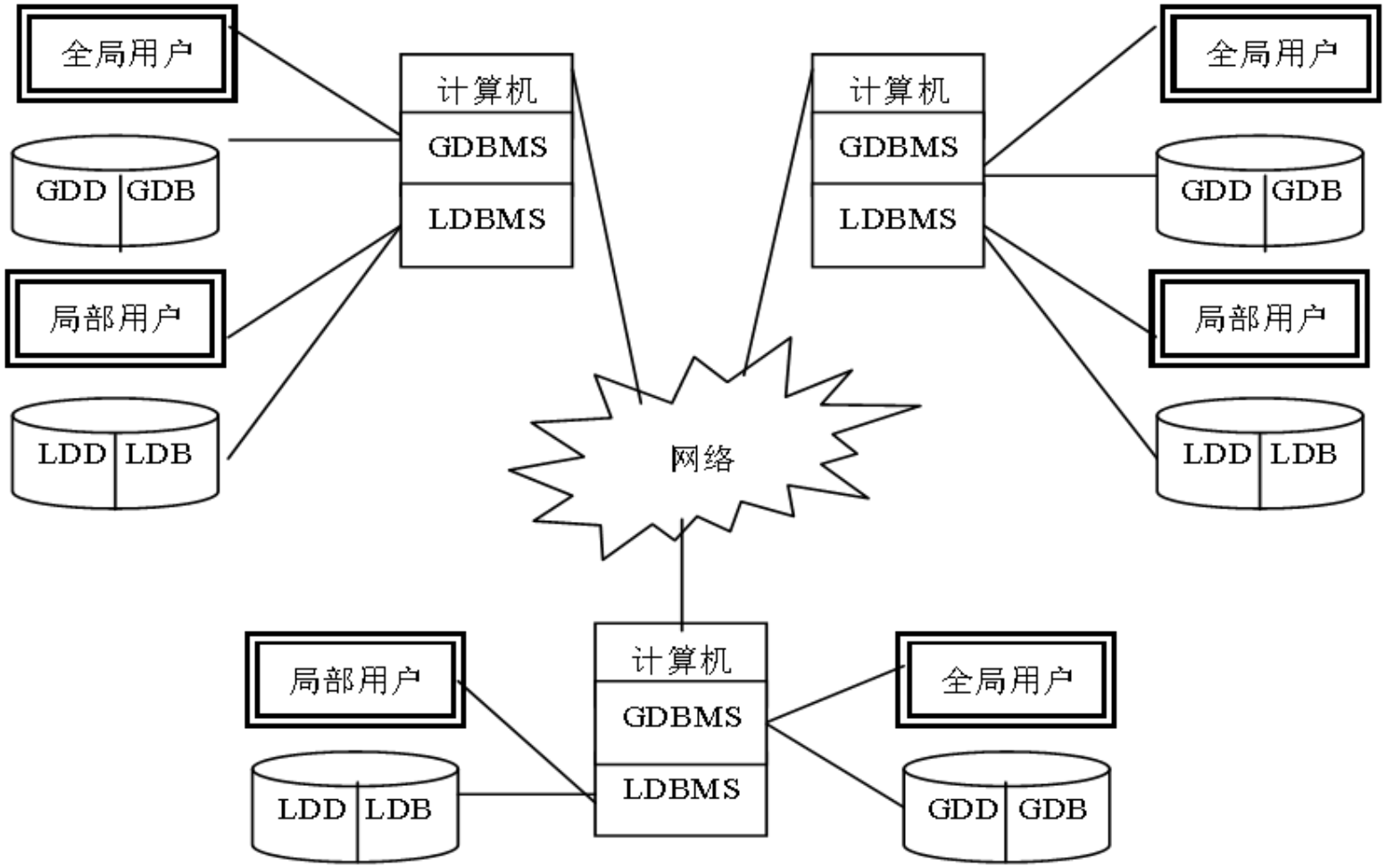
10.1 概述

➢分布式数据库系统的组成



10.1 概述

➢分布式数据库系统的组成



10.1 概述

➢分布式数据库系统的特点

- 物理分布性：数据不是存储在一个场地上，而是存储在计算机网络的多个场地上。
- 逻辑整体性：数据物理分布在各个场地，但逻辑上是一个整体，它们被所有用户(全局用户)共享，并由一个DDBMS统一管理。
- 场地自治性：各场地上的数据由本地的DBMS管理，具有自治处理能力，完成本场地的应用(局部应用)。
- 场地之间协作性：各场地虽然具有高度的自治性，但是又相互协作构成一个整体。

10.1 概述

➢分布式数据库系统的分类

- 同构同质型DDBS：各个场地都采用同一类型的数据模型(譬如都是关系型)，并且是同一型号的DBMS。
- 同构异质型DDBS：各个场地采用同一类型的数据模型，但是DBMS的型号不同，譬如DB2、ORACLE、SYBASE、SQL Server等。
- 异构型DDBS：各个场地的数据模型不同，甚至类型也不同。

10.2 分布式数据存储

➢分布式数据存储

- 在分布式数据库中存储关系r的几种方法：
  - ✓数据复制
  - ✓数据分片

10.2 分布式数据存储

➢数据复制

- 数据复制：系统维护关系r的几个完全相同的副本(拷贝)，各个副本存储在不同的节点上。
- 全复制：系统中的每个节点都存有关系r的一个拷贝。
- 主副本：指定关系r的多个副本中的一个作为主副本，从而简化副本管理。

10.2 分布式数据存储

➢数据复制的优缺点

- 可用性增强：当包含关系r的节点之一发生故障时，关系r可以在另外的节点上找到。
- 并行度增大：几个节点就可以并行地处理涉及关系r的查询。并可以减少数据在节点间的移动。
- 更新开销增大：对关系r的更新必须传播到包含r的副本的所有节点。

10.2 分布式数据存储

➢数据分片

- 数据分片：将关系r划分为多个片段r_1，r_2，…，r_n。这些片段中包含足够的信息，使得能够重构原始关系r
- 水平分片：将关系r划分为多个子集：

$$r_1，r_2，…，r_n$$

r的每个元组必须至少属于一个片段。

- 垂直分片：将关系r(R)投影到R的属性的多个子集R_1，R_2，…，R_n。为保证关系r能被重构需要在每个R_i中都包含R的主码属性，或特殊的tuple-id属性。

10.2 分布式数据存储

➢数据分片

- 导出分片：又称为导出水平分片

导出分片是指导出水平分片，即水平分片的条件不是本身属性的条件而是其他关系的属性的条件。例如学生选课关系 SC(Sno，Cno，Grade)，若不是按照学号或课程号或成绩的某类条件分片，而是按照学生年龄＞18 岁和≤18 岁来分片(学生年龄是学生关系 Student 的属性)，则这类分片称导出分片，可以用 SQL 语句来表达这两个片段。

年龄＞18 岁的学生选课片段 SC_A(Sno，Cno，Grade)由下面的查询结果组成：

```
SELECT  Sno，Cno，Grade
FROM  S，SC
WHERE  S.Sno=SC.Sno  AND  S.Sage＞18；
```

年龄≤18 岁的片段 SC_B 由下面的查询结果组成：

```
SELECT  Sno，Cno，Grade
FROM  S，SC
WHERE  S.Sno=SC.Sno  AND  S.Sage≤18；
```

- 混合分片：在水平分片或垂直分片的结果上再进行垂直分片或水平分片。

10.2 分布式数据存储

➢数据分片的条件

- 完备性条件：必须把全局关系的所有数据映射到片段中，不允许有属于全局关系的数据却不属于它的任何一个片段。
- 可重构条件：必须保证能够由同一个全局关系的各个片段来重建该全局关系。对于水平分片可用并操作重构全局关系；对于垂直分片可用连接操作重构全局关系。
- 不相交条件：要求一个全局关系被分割后所得的各个数据片段互不重叠(对垂直分片的主键除外)。

10.2 分布式数据存储

➢水平分片

- 将关系r按行分为若干子集r_1，r_2，…，r_n，每个子集r_i称为一个水平片段。
- 一个水平片段可以看成是关系上的一个选择

$$\gamma_i = \sigma_{P(i)}(r)$$

- 如$M_s = \sigma_{Sex='M'}(S)$
- 关系的重构可以通过并运算来实现。

$$r = r_1 \cup r_2 \cup \ldots \cup r_n$$

10.2 分布式数据存储

➢垂直分片

- 将关系r按列分为若干属性子集r_1，r_2，…，r_n，每个子集r_i称为一个垂直片段。
- 一个垂直片段可以看成是关系上的一个投影：$\gamma_i = \prod_{R_i}(\gamma)$

  其中R_i是r的一个属性子集，如：$P_S = \prod_{SNO,SNAME}(S)$
- 关系的重构可以通过连接运算来实现：

$$r = r_1 \bowtie r_2 \bowtie \ldots \bowtie r_n$$

要求所有垂直分片的片段都包括关系的主键。

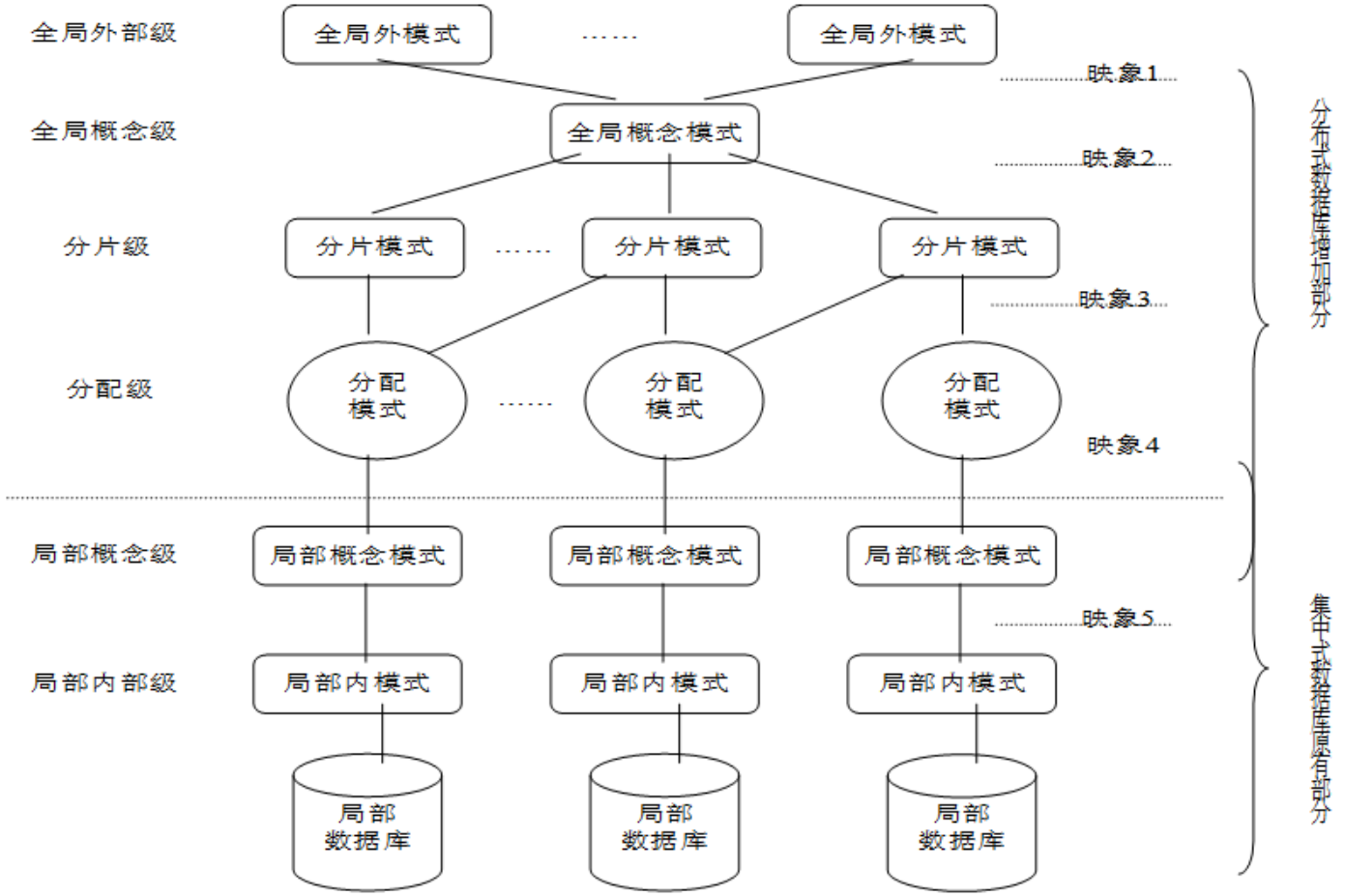
10.2 分布式数据存储

➢分片的优点

- 水平:
  - ✓ 允许对关系的各片断做并行处理。
  - ✓ 关系划分时可使元组分配到最频繁使用该元组的场地上。
- 垂直:
  - ✓ 分解属性时可使不同属性存储在最频繁使用的场地上。
  - ✓ tuple-id 属性使得垂直片断之间可以高效连接。
  - ✓ 允许对关系作并行处理。
- 水平与垂直分片可以混合
  - ✓ 可以连续进行分片到任意深度。

10.3 分布式数据库系统体系结构

➢分布式数据库系统体系结构



10.3 分布式数据库系统体系结构

➢体系结构

① 全局外模式

② 全局概念模式

③ 分片模式

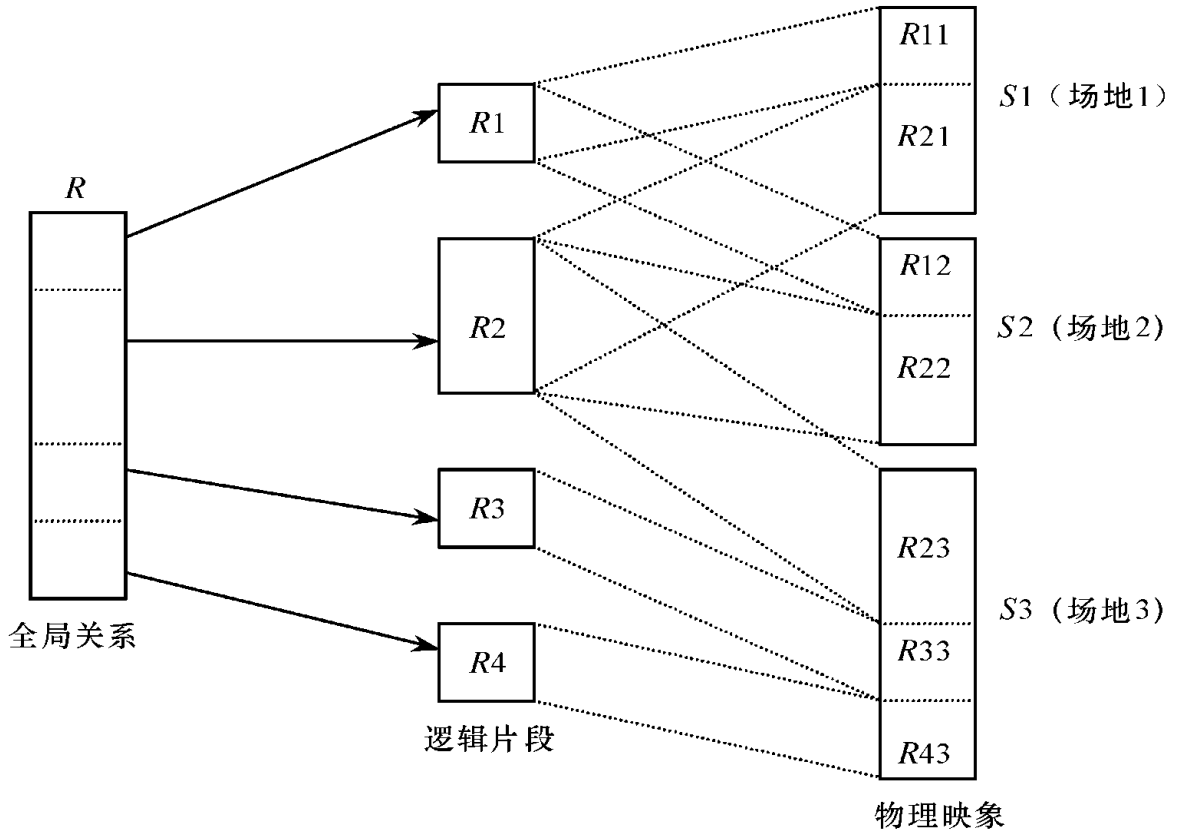
④ 分配模式

⑤ 局部概念模式

⑥ 局部内模式

10.3 分布式数据库系统体系结构

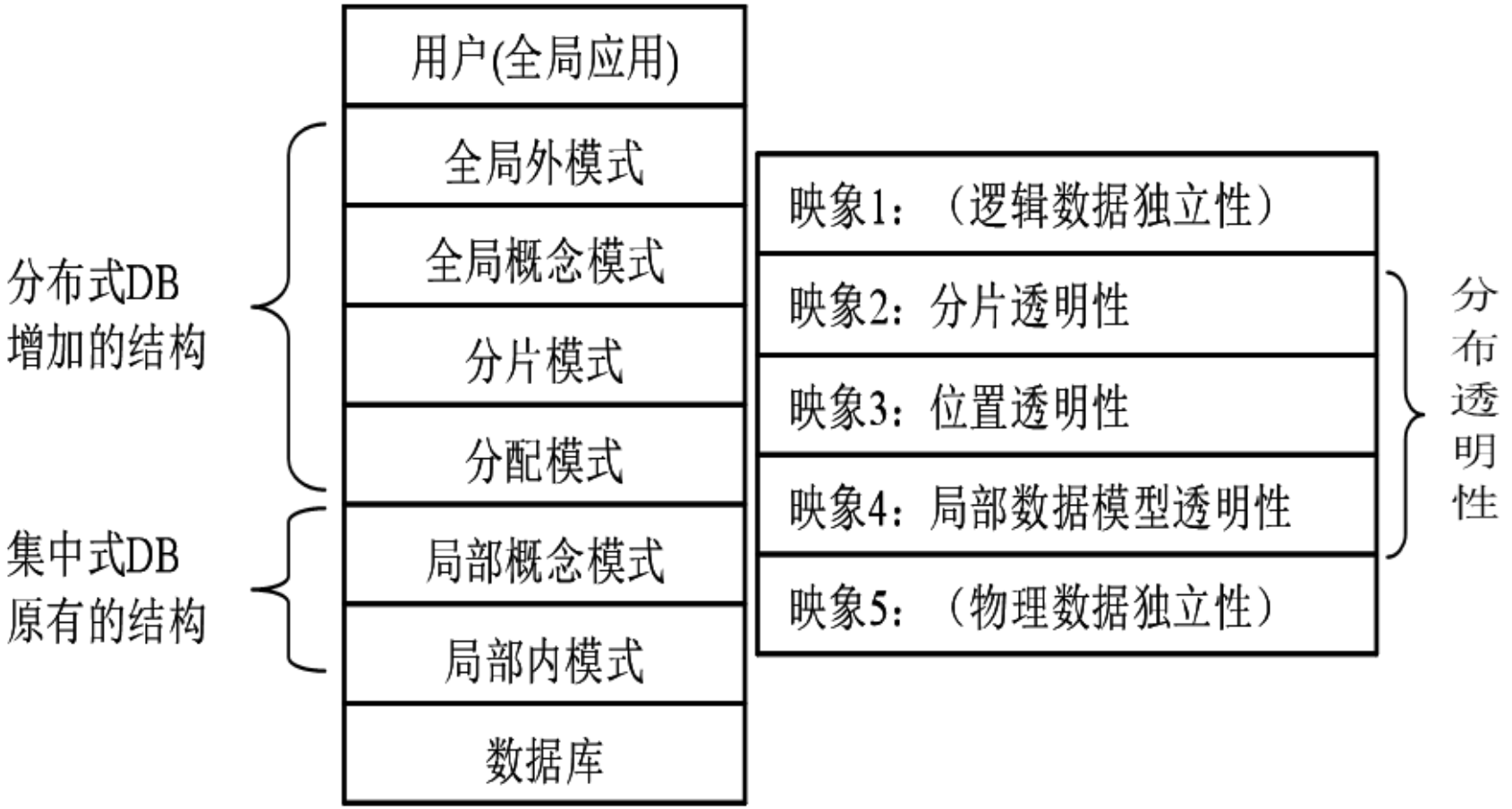
➢举例



全局关系R的逻辑片段与物理映象

10.3 分布式数据库系统体系结构

➢分布透明性



10.3 分布式数据库系统体系结构

➢分布(网络)透明性

- 系统向用户屏蔽网络中数据分布的细节
  - ✓副本透明性
  - ✓分片透明性
  - ✓位置透明性
- 问题:
  - ✓ 数据项的命名和目录管理
  - ✓ 透明性与更新

10.3 分布式数据库系统体系结构

➢分布(网络)透明性

- 分片透明性(全局视图和分片视图之间)

用户或应用程序只对全局关系进行操作而不必考虑关系的分片。如果分片模式改变了，通过调整全局模式与分片模式之间的映象关系来保持全局模式不变。

- 位置透明性(分片视图和分配视图之间)

用户或应用程序不必了解片段的存储位置

- 局部数据模型透明性

(分配视图和局部概念视图之间)用户或应用程序不必了解局部场地上使用的是哪种数据模型。

10.4 分布式数据库管理系统

➢分布式数据库管理系统

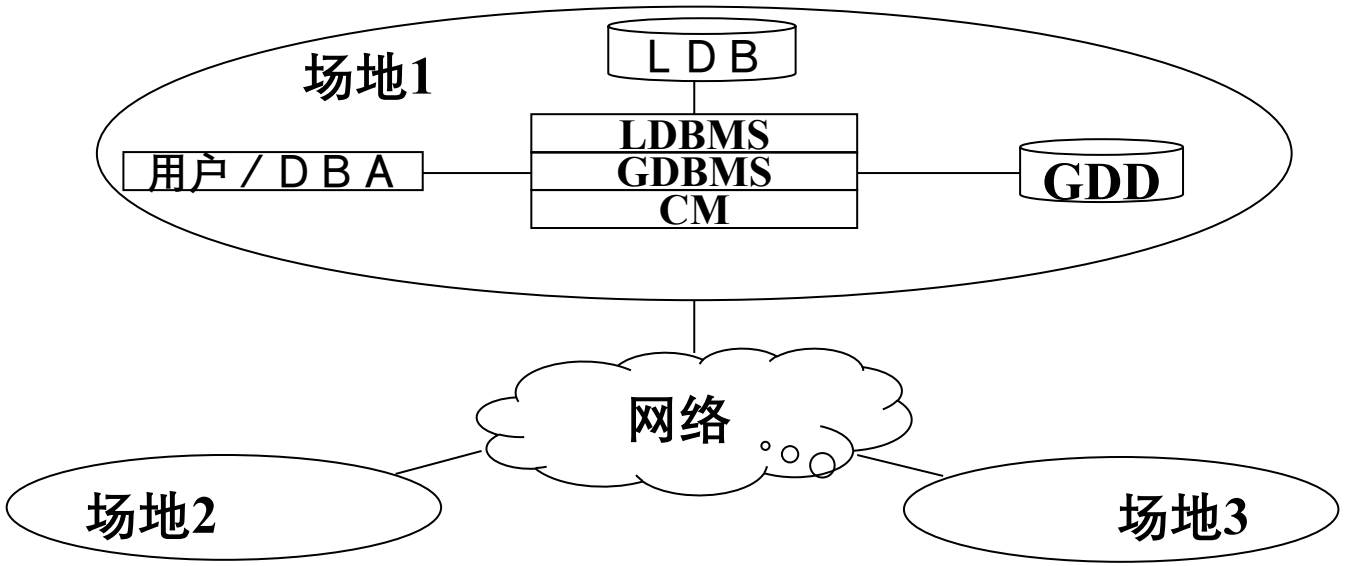
- DDBMS是一种典型的实现分布式数据管理的软件系统，用于支持分布式数据库的创建、运行、管理和维护。
- DDBMS能对分布于各个场地的软、硬件资源进行统一管理和控制，使其在逻辑上可视为一个整体的数据库系统，并为用户提供与分布式数据的接口。
- 同时，由于数据的分布性，在管理机制上还必须具有计算机网络通信协议的分布管理特性。

10.4 分布式数据库管理系统

➢分布式数据库管理系统的组成

DDBMS一般包括4个部分：

- ✓全局数据库管理系统GDBMS(Global DBMS)
- ✓全局数据字典GDD(Global Data Dictionary)
- ✓局部数据库管理系统LDBMS(Local DBMS)
- ✓通信管理CM(Communication Management)



DDBMS结构图

10.4 分布式数据库管理系统

➢分布式数据库管理系统的组成

- 全局数据库管理系统GDBMS

GDBMS是DDBMS的核心，负责提供分布透明性，协调全局事务的执行及协调各场地上的LDBMS共同完成全局应用，GDBMS通常包括:

(1)用户接口层：提供一个用于检验用户身份的接口，用户的应用程序经用户接口处理，作为一个全局事务由DDBMS执行。

(2)语言处理层：负责查询语言的语法、词法分析，把查询语句转换成某种内部表示形式，如用语法树表示查询。

10.4 分布式数据库管理系统

➢分布式数据库管理系统的组成

- 全局数据库管理系统GDBMS

(3)分布式数据管理层

主要完成查询分解、优化和确定查询计划。

(4)分布式事务管理层

用于对分布式事务进行并发控制，并提供全局恢复功能。

(5)全局数据与局部数据之间转换层

✓对异构系统，需将数据转换成系统可接受的形式

✓具体的转换有：数据模型的转换；数字代码格式、字长、精度、单位等的转换；操作命令、完整性规则、安全性规则的转换等。

10.4 分布式数据库管理系统

➢分布式数据库管理系统的组成

- 全局数据字典GDD
  - GDD负责提供系统的各种描述、管理和控制信息。如为系统提供各级模式描述、网络描述、存取权限、事务优先级、完整性约束与相容性约束、数据的分割及其定义、副本数据及其所在场地、存取路径、死锁检测、预防及故障恢复，与数据库运行质量有关的统计信息等。
  - 由于数据是分布的，因此数据字典也存在一个分布策略及管理问题 ,数据字典中的数据与冗余也需要进行优化。

10.4 分布式数据库管理系统

➢分布式数据库管理系统的组成

- 局部数据库管理系统LDBMS

  LDBMS用来建立和管理各场地上的局部数据库LDB，提供场地的自治能力，可执行局部应用和全局查询的子查询。

- 通信管理CM

  CM遵循网络协议，实现各场地之间数据的可靠传送，完成系统的通信功能。

10.4 分布式数据库管理系统

➢分布式数据库管理系统的分类

- 按LDBMS类型分类

(1)同构型DDBMS(Homogeneous DDBMS)

同构型(也称均质型)系统是指系统中每个场地的LDBMS类型都相同，即它们支持相同的数据模型、访问方法、优化策略、并发控制算法，以及相同的命令语言和查询语言等。

(2)异构型DDBMS(Heterogeneous DDBMS)

异构型(也称异质型)系统是指系统中每个场地可以有不同类型的LDBMS。它们可以是不同的数据模型，也可以是同一数据模型但由不同厂商提供的产品。

10.4 分布式数据库管理系统

➢分布式数据库管理系统的分类

- 按全局控制方式分类

(1)全局集中控制的DDBMS

这种方式的GDBMS和GDD集中在一个中心场地上，由该场地完成全局事务的协调和局部数据库转换等一切控制问题。

(2)全局控制分散的DDBMS

GDBMS和GDD分散存放在各个场地上，即每个场地都有控制信息的一个副本，都能完成全局事务的协调和局部数据库转换的控制问题。

(3) 全局控制部分分散的DDBMS

这种方式根据应用的需要将GDBMS和GDD分散在某些场地上，即只在部分场地包含控制信息的副本，性能介于以上两种类型之间。

10.4 分布式数据库管理系统

➢分布式数据库管理系统的分类

- 按数据在逻辑上集中/分布分类：

(1)物理上分布、逻辑上集中的DDBMS

这种方式数据库在逻辑上是集中的，用户感觉不到数据的分布，看到的似乎是一个数据模式为全局数据模式的集中式数据库。

(2)物理上分布、逻辑上分布的DDBMS

✓这种分布式数据库系统又称联邦分布式数据库系统(Federated Distributed Database Systems)

✓系统中每个场地所看到的数据模式是不同的，好像有多个逻辑数据库存在。

10.4 分布式数据库管理系统

➢分布式数据库管理系统的分类

- 按构造方式分类

(1)综合型体系结构

设计一个全新的DDBMS时，设计人员可综合权衡用户需求，采用自顶向下的设计方法，设计一个完整的DDBMS，然后把系统的功能按一定的策略分散配置在一个分布式环境中。

(2)联合型体系结构

在原有DBMS基础上建立分布式DBMS，按照使用LDBMS类型的不同又分为同构型和异构型DDBMS。

10.5 分布式查询处理与优化

➢分布式查询实例

- 例：一个简化的供应商-零件分布数据库

S(Sno，City)　　10^4个元组，存放在场地A；
P(Pno，Color)　　10^5个元组，存放在场地B；
SP(Sno，Pno)　　10^5个元组，存放在场地A；
设每个关系的元组均为100字节长。

- 查询：查询北京的，并供应红色零件的供应商号

```
SELECT  S.Sno
FROM  S，P，SP
WHERE S.City='北京'  AND
          SP.Pno=P.Pno  AND SP.Sno=S.Sno AND
          P.Color='红色';
```

10.4 分布式查询处理与优化

➢分布式查询实例

- 估算值(某些中间结果的元组数)

红色零件数=10

北京供应商的装运单数=10^5

- 对通信系统的假定

数据传输速度= 10^4字节/秒

传输延迟=1秒

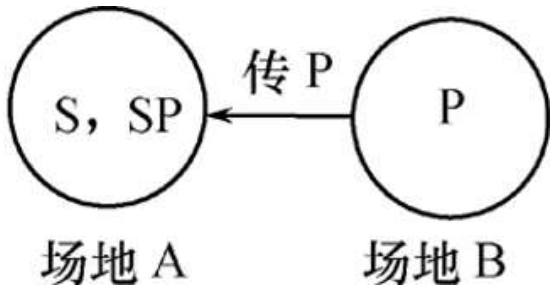
10.5 分布式查询处理与优化

➢分布式查询实例

- 6种可能的查询存取策略，对每种i分别计算通信时间T[i]：

T[i] = 总传输延迟+总数据量/数据传输速度(单位：b/s)

- 策略1 把关系P传送到场地A，在A地进行查询处理

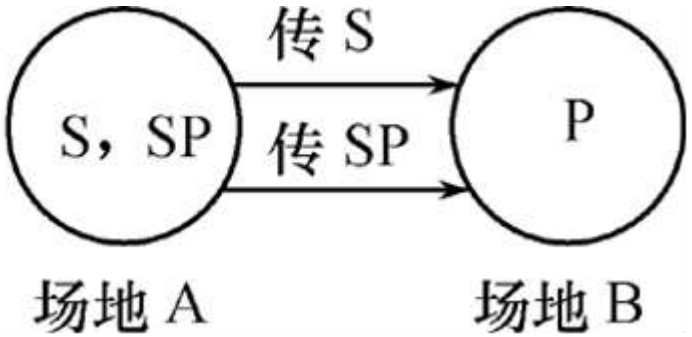


- T[1]=1+ 10^5 ×100/ 10^4 =103秒(16.7分)

10.5 分布式查询处理与优化

➢分布式查询实例

- 策略2 把关系S、SP传到场地B，在B地执行查询处理
-

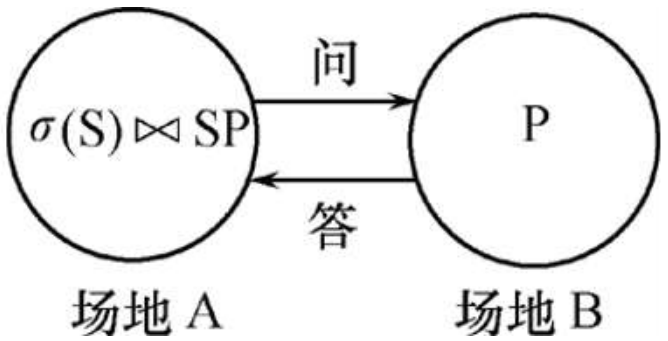


- T[2]=2+(10^4 + 10^6)×100/ 10^4 ≈10100秒(2.8小时)

10.5 分布式查询处理与优化

➢分布式查询实例

- 策略3:在场地A连接关系S和SP，选出城市为北京的元组(10^5个)，然后对这些元组中的每个元组的Pno，询问场地B，看此零件是否红色
- 共问答10^5次，由于不是传送数据，只是消息的问答
- T[3] ≈ 2× 10^5 s (2.3天!)

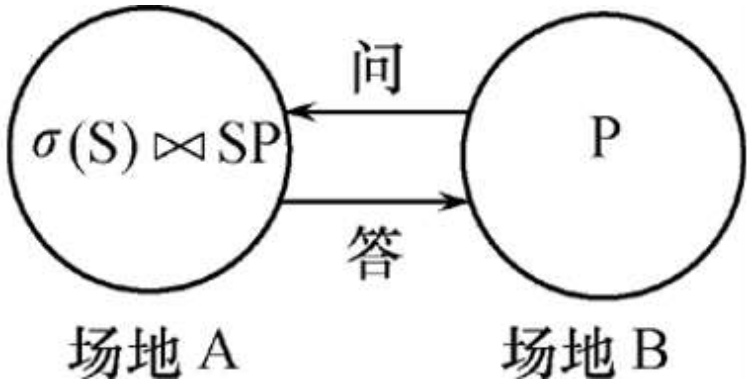


10.5 分布式查询处理与优化

➢分布式查询实例

- 策略4　在场地B选出红色零件的元组(10个)，然后对每一个元组逐一检查场地A，看北京供应商的装运单中是否有这个零件装运单(若有则选出S#)
- 每做这样一次检查包括2次消息，共问一答10次

T[4]=2×10=20秒

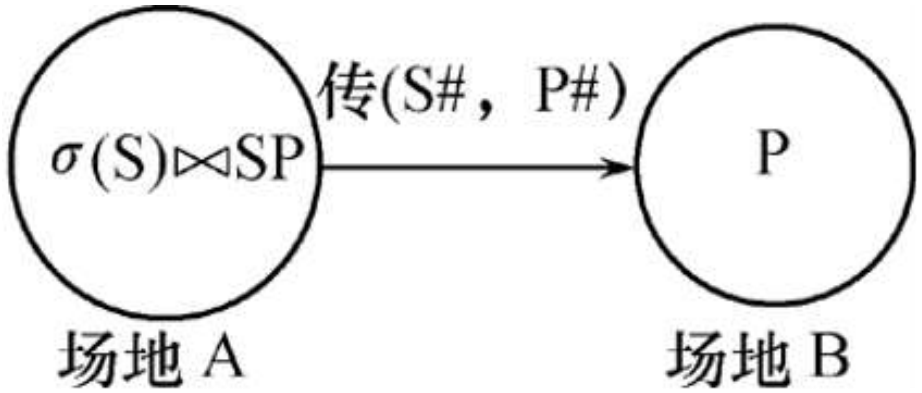


10.5 分布式查询处理与优化

➢分布式查询实例

- 策略5 在场地A选出北京的供应商的装运单(10^5) 把结果送到场地B，在场地B完成最后处理，所以

$$T[5]=1+(10^5 \times 100)/\ 10^4 \approx 1000\text{秒}(16.7\text{分})$$

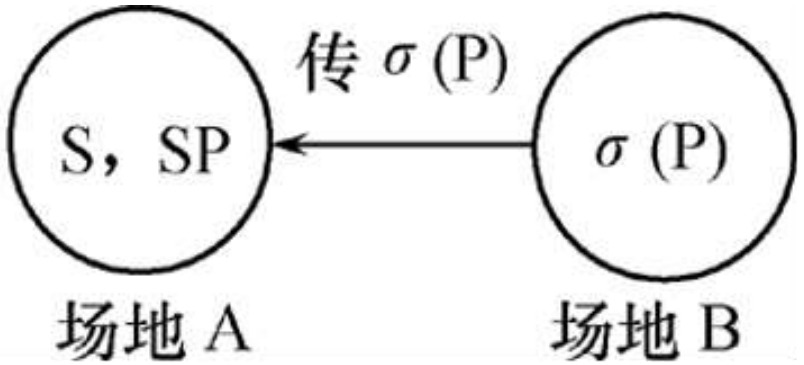


10.5 分布式查询处理与优化

➢分布式查询实例

- 策略6：在场地B的关系P中选出红色的元组(10个)，把结果送到场地A完成最终处理。所以

$$T[6]=1+(10\times100)/\ 10^4\ \approx 1秒$$



10.5 分布式查询处理与优化

➢分布式查询实例

策略	通信时间	方　法
1	16.7分	把P传到场地A
2	2.8小时	把S，SP传到场地B
3	2.3天	对每一北京的装运单，检查相应零件是否红色
4	20秒	对每一红色零件，检查北京供应商中是否有人供应
5	16.7分	把北京供应商的装运单传送到场地B
6	1秒	把红色零件传送到场地A

10.5 分布式查询处理与优化

➢分布式查询实例

- (1) 不同的存取策略通信时间相差很大，达多个数量级！ -- 因此需要优化。
- (2)不同策略，不同的考虑方式。
  - ✓有些策略中数据传输速度和传输延迟都要考虑
  - ✓有些策略中(如策略3、策略4)主要考虑传输延迟
  - ✓有些策略中(如策略1、策略2、策略5)数据传输量大，主要考虑传输时间

10.5 分布式查询处理与优化

➢分布式查询处理和优化要解决的问题

- 分布式数据库系统中的三类查询：
  - ✓局部查询
    - ■本地查询
    - ■远程查询
  - ✓全局查询
- 局部查询和远程查询优化
  - ✓采用集中式数据库的查询优化技术(代数优化和物理优化)

10.5 分布式查询处理与优化

➢分布式查询处理和优化要解决的问题

- 全局查询处理和优化涉及的问题
  - ✓1.查询分解
  - ✓2.查询本地化
  - ✓3.选择操作执行的次序
  - ✓4.选择执行操作的方法

10.5 分布式查询处理与优化

➢分布式查询优化的代价

- 集中式数据库的查询代价
  - ✓I/O代价+CPU代价
  - ✓首要优化目标： I/O代价最少
- 分布式数据库的查询开销
  - ✓I/O代价+CPU代价+通信代价
  - ✓首要优化目标：通信代价最少

10.5 分布式查询处理与优化

➢分布式查询优化的代价

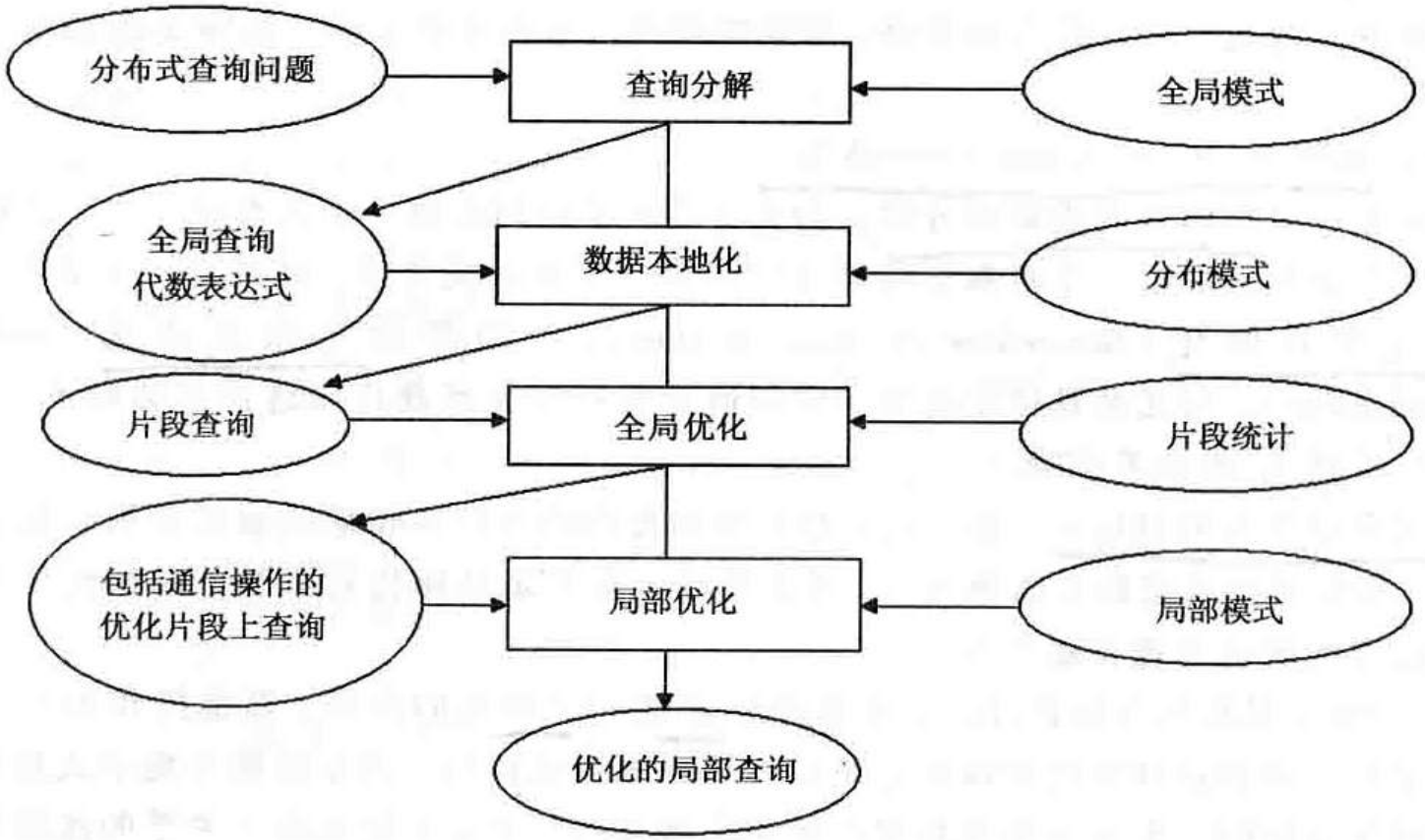
- 通信代价可以用下面的公式粗略计算：

$$TC(X) = C0 + X * C_1$$

- X：数据传输量，以b(位)为单位计算；
- C_0 ：两结点之间初始化一次传输所花费的开销，它由通信系统决定，近似为一个常数，单位为s(秒)；
- C_1 ：单位数据(b)传输的代价，单位为(s/b)

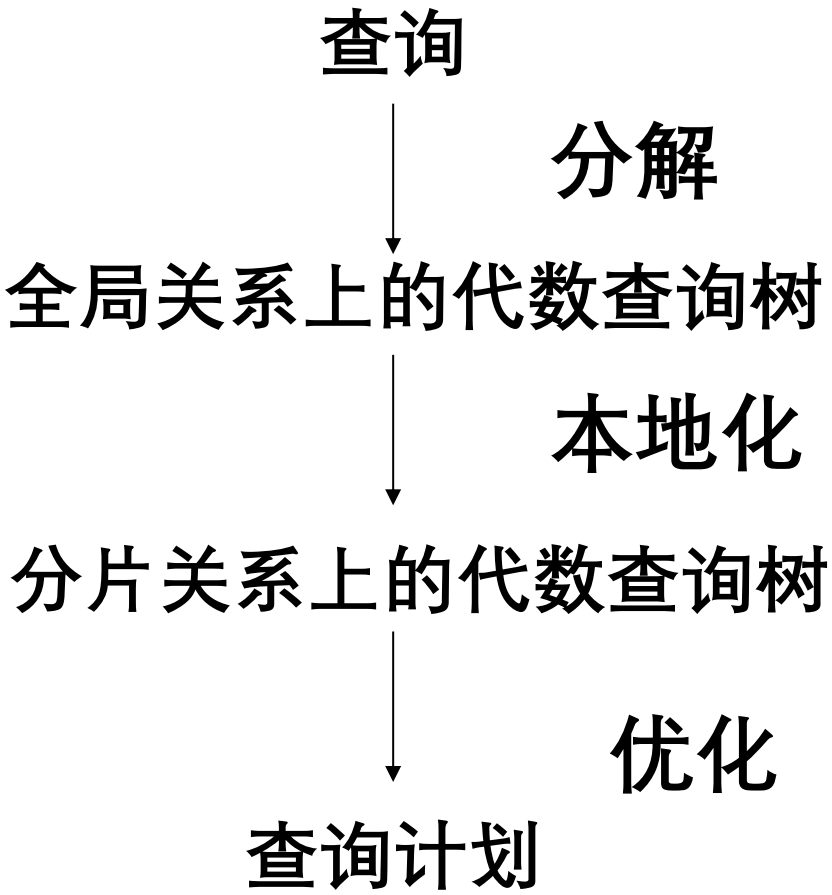
10.5 分布式查询处理与优化

➢分布式查询处理的层次结构



10.5 分布式查询处理与优化

➢分布式查询处理的层次结构



10.5 分布式查询处理与优化

➢查询分解(Query Decomposition)

- 查询分解是将查询问题，转换成一个定义在全局关系上的关系代数表达式。
- 这一层的做法与集中式DBMS相同，因为并未涉及到分布问题。
- 本层转换所需要的信息在全局概念模式中获得。

10.5 分布式查询处理与优化

➢查询分解(Query Decomposition)

- 技术
  - ✓代数转换
  - ✓消除冗余
  - ✓代数重写

10.5 分布式查询处理与优化

➢查询分解(Query Decomposition)

- 例子：条件矛盾或蕴含

$$(E.sal=2000) \wedge (E.sal>5000)$$

$$\Rightarrow \text{ False}$$

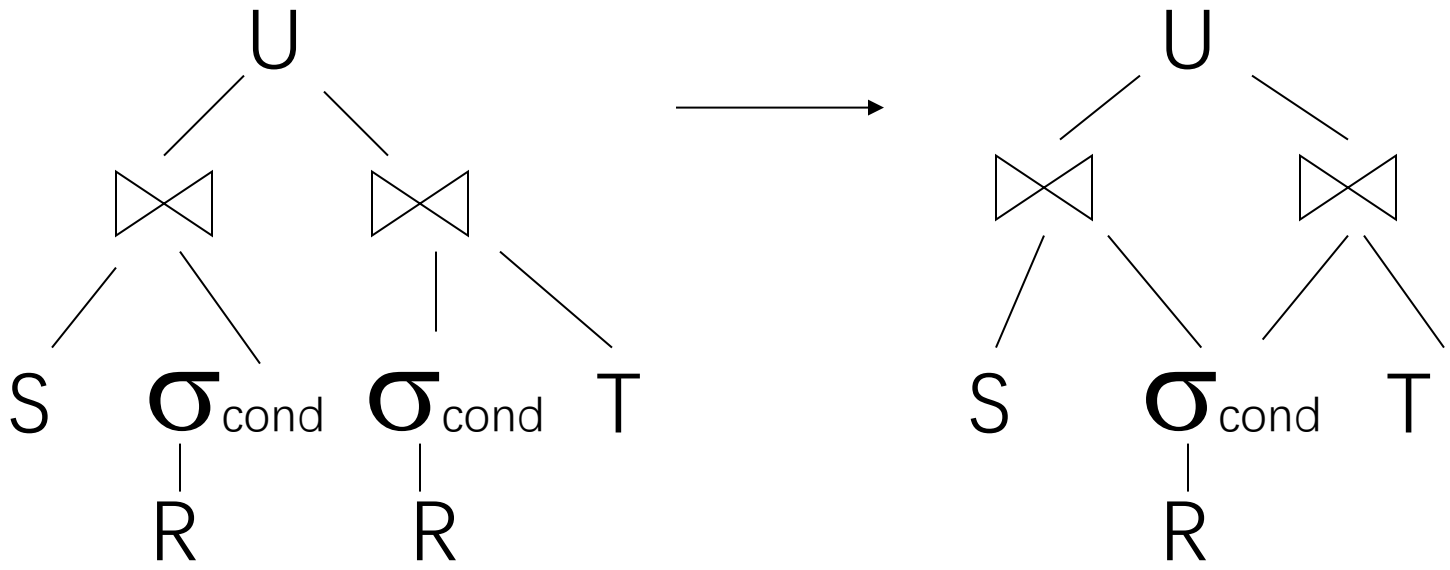
$$(E.sal<5000) \wedge (E.sal<3000)$$

$$\Rightarrow E.sal<3000$$

10.5 分布式查询处理与优化

➢消除冗余

- 例子：公共子表达式



10.5 分布式查询处理与优化

➢数据本地化(Data Localization)

- 数据本地化是把一个在全局关系上的查询，进行具体化，落实到适合的(尽可能做到本地化或近地化)片段上的查询。
- 即，将在全局关系上的关系代数表达式，变换为在相应片段上的关系代数表达式。
- 这一变换所需要的信息在分片模式和片段的分配模式中获得。

10.5 分布式查询处理与优化

➢数据本地化(Data Localization)

- 技术:
  - ✓使用查询树描述
  - ✓从全局查询映射到片段查询
  - ✓上推 $\cup$
  - ✓下推π,σ
  - ✓消除不必要的操作

10.5 分布式查询处理与优化

➢数据本地化举例

例：设有下列全局关系和分片

全局关系 **SUPPLIER (SNO, SNAME, CITY)**

水平分片:

SUPPLIER1 = $\sigma_{CITY=\text{'北京'}}$ (SUPPLIER)
SUPPLIER2 = $\sigma_{CITY=\text{'上海'}}$ (SUPPLIER)
SUPPLIER3= $\sigma_{CITY=\text{'广州'}}$ (SUPPLIER)

全局关系**PROJECT(PNO, PNAME, MNG, DEPT)**

水平分片:

PROJECT 1 = $\sigma_{DEPT=\text{'教育厅'}}$ (PROJECT)
PROJECT 2 = $\sigma_{DEPT=\text{'科技厅'}}$ (PROJECT)
PROJECT 3= $\sigma_{DEPT=\text{'科工委'}}$ (PROJECT)

全局关系**SUPPLY(SNO, PNO, MNO)**

10.5 分布式查询处理与优化

➢数据本地化举例

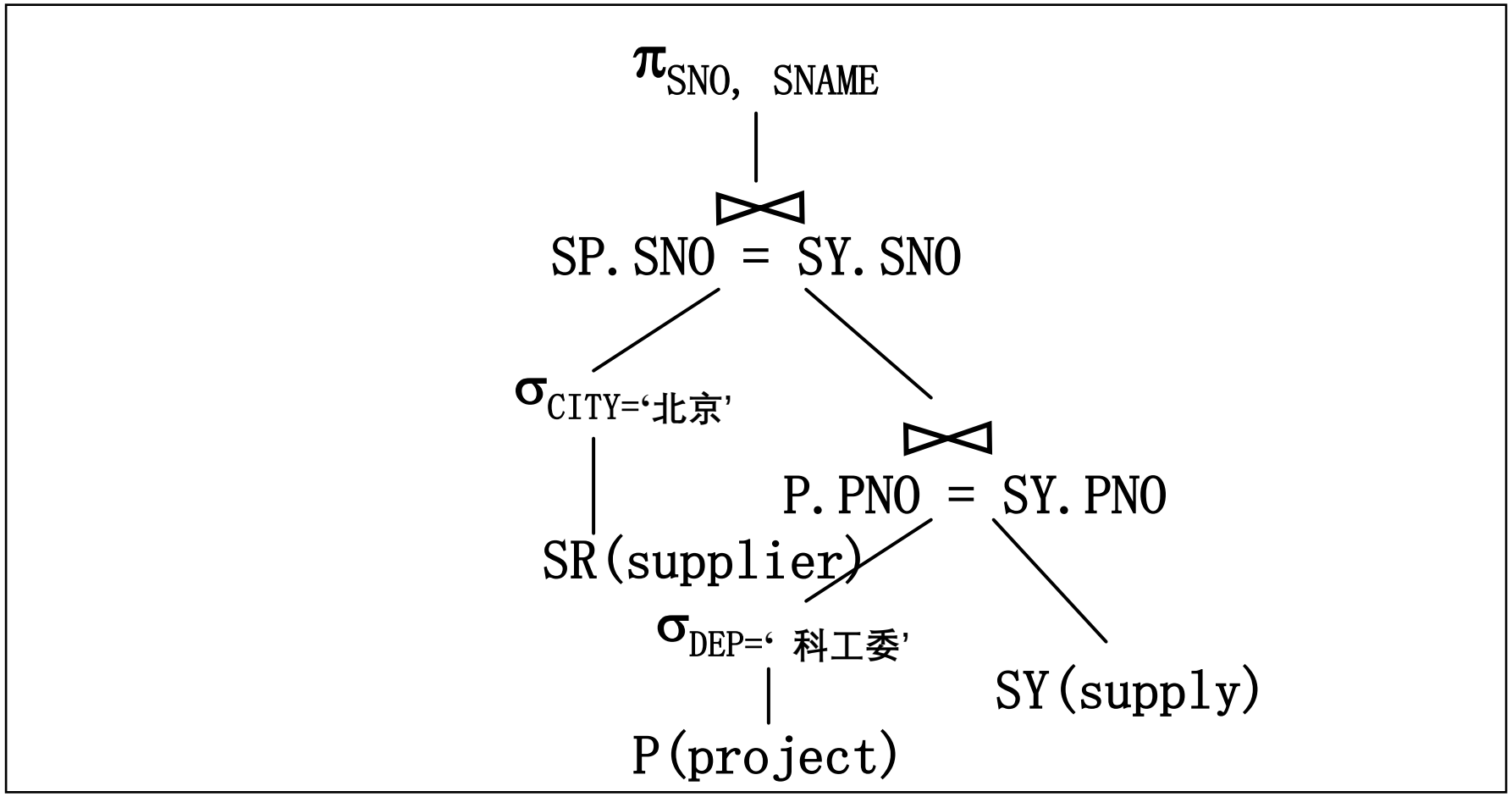
例：查询给”科工委”主管的工程供应材料的北京的供应商编号和名称,全局查询表示如下：

$$\Pi_{SNO,\ SNAME}\ (\sigma_{CITY=\ '北京'}\ (SUPPLIER)\ \bowtie$$
$$(\ \sigma_{\ DEPT\ =\ '科工委'}\ (PROJECT)\ \bowtie SUPPLY))$$

10.5 分布式查询处理与优化

➢数据本地化举例

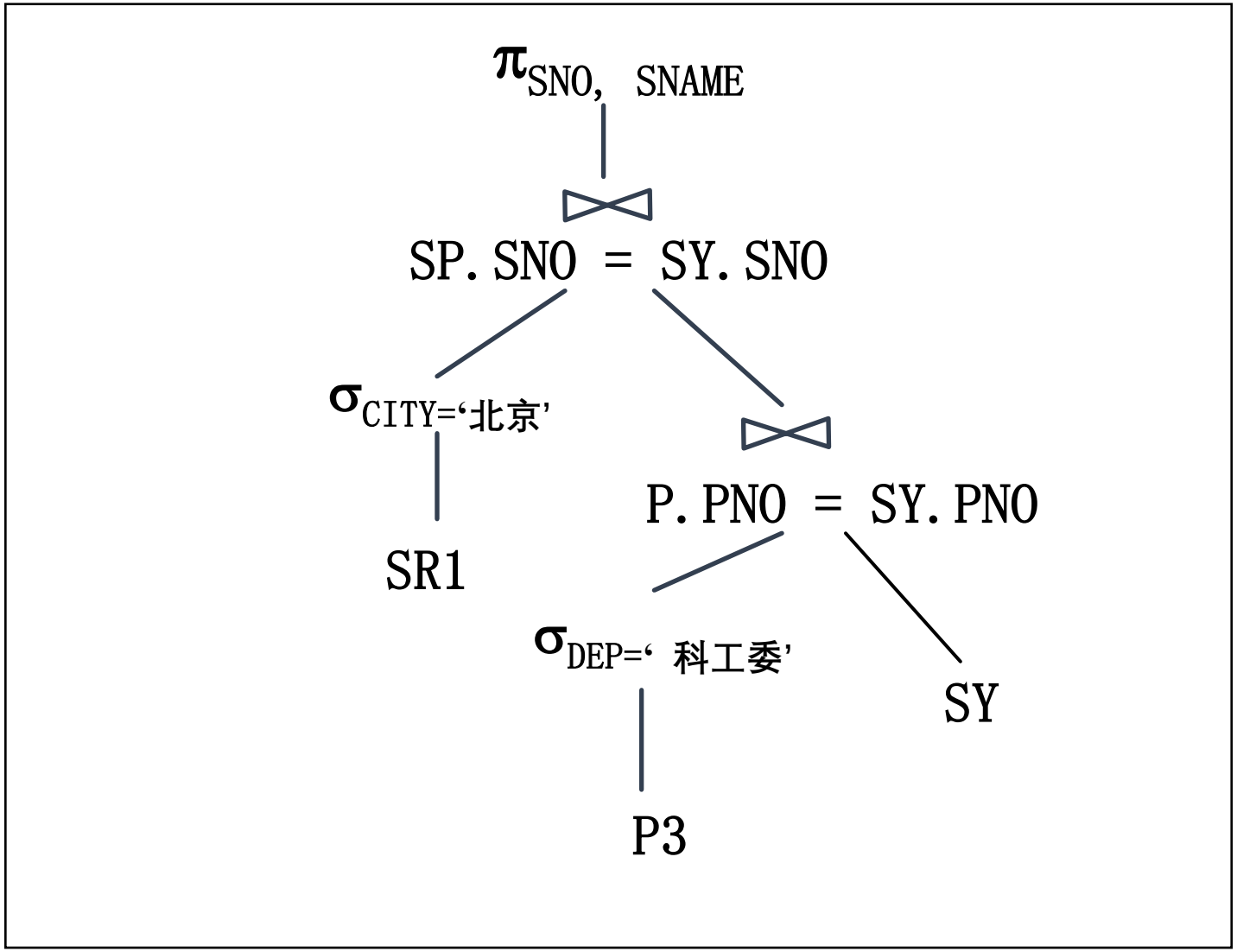
全局查询树:



10.5 分布式查询处理与优化

➢数据本地化举例

全局查询到片段查询映射



10.5 分布式查询处理与优化

➢全局优化(Global Optimization)

- 全局优化的输入是分片查询, 即在片段上的查询。
- 查询优化的目标在于寻找一个近于最优的执行策略。
- 前面两层已经通过消除冗余表达式，查询本地化或近地化等做了某些优化。
- 全局优化即是找出分片查询的最佳操作次序，使得代价函数最小。

10.5 分布式查询处理与优化

➢全局优化(Global Optimization)

- 全局优化需解决的主要问题：

(1) 多副本的选择：

(2) 执行次序的选择：　主要确定多元连接的连接次序

(3) 连接方法的选择：　嵌套循环、合并扫描、半连接

(4) 执行结点的选择：

10.5 分布式查询处理与优化

➢局部优化(Local Optimization)

- 最后一层局部查询优化由拥有与查询有关的片段的各个场地执行。
- 在每一个场地上执行的子查询，被称为局部查询。
- 局部查询由该场地上的DBMS进行优化，采用集中式系统算法，所需信息取自局部模式。

10.6 分布式事务处理

➢分布式事务管理

- 在分布式数据库环境中，一个数据库事务可以更新多个场地上的数据，这种数据库事务称为分布式事务。
- 分布式事务必须满足传统事务的ACID特性，即原子性，一致性，分离性和持久性。

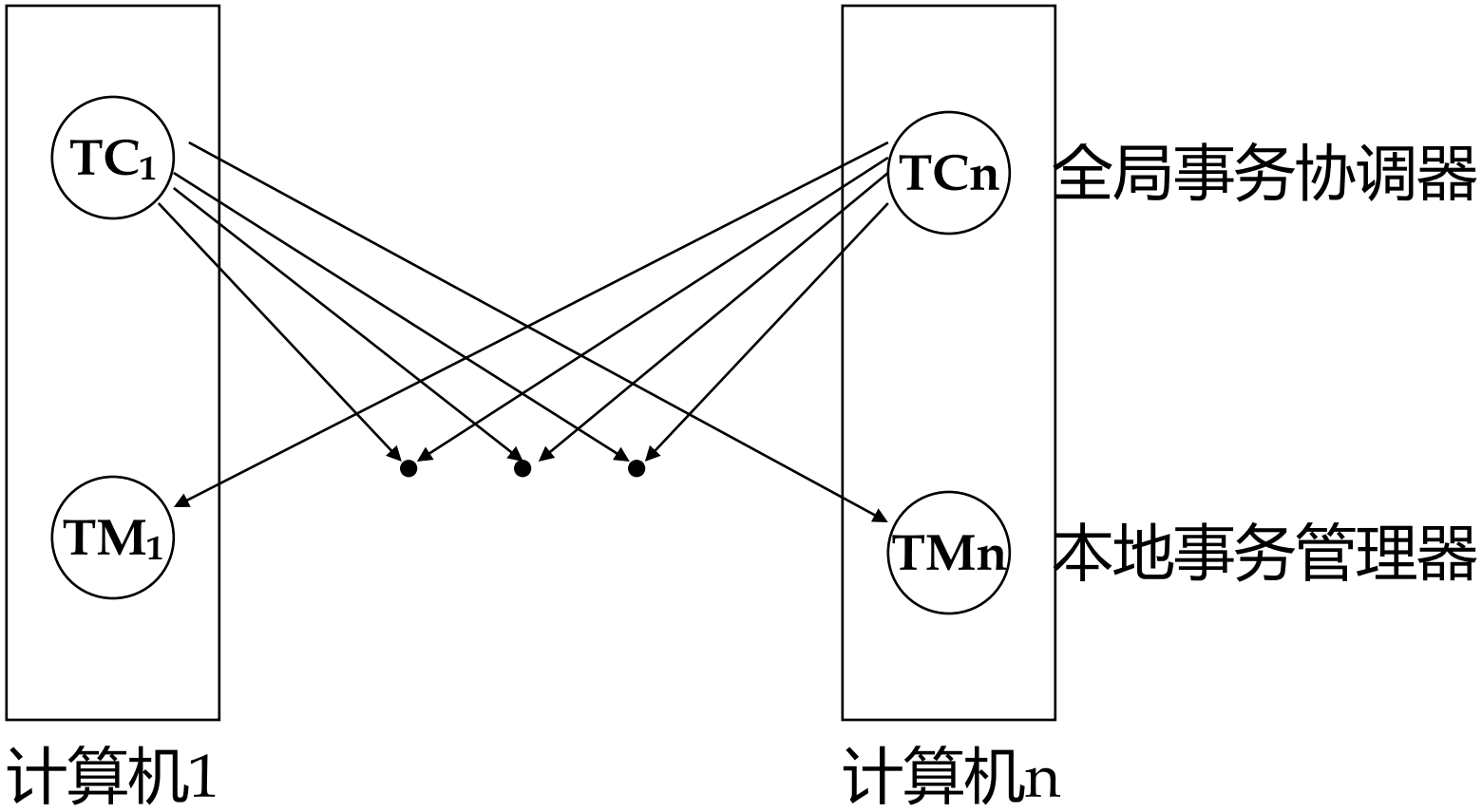
10.6 分布式事务处理

➢分布式事务管理全局事务和局部事务

- 全局事务：访问和更新多个局部数据库中数据的事务。
- 局部事务：仅访问和更新一个局部数据库中数据的事务。

10.6 分布式事务处理

➢分布式事务管理抽象模型



10.6 分布式事务处理

➢分布式事务管理抽象模型

- 本地事务管理器：管理那些访问存储在一个局部节点中数据的事务(或子事务)的执行。这样的事务既可以是局部事务，也可以是全局事务的一部分。
- 负责
  - ✓维护一个用于恢复的日志。
  - ✓参与适当的并发控制模式，以协调在该节点执行的事务的并发执行。

10.6 分布式事务处理

➢分布式事务管理抽象模型

- 全局事务协调器:协调该节点上发起的各事务(局部事务和全局事务)的执行。
- 负责
  - ✓启动事务的执行。
  - ✓将全局事务分解为子事务，并将这些子事务分派到恰当的节点上去执行。
  - ✓协调事务的终止，其结果就是要保证全局事务在所有节点上都提交或在所有节点上都中止。

10.6 分布式事务处理

➢提交协议与故障恢复

- 为保证全局事务的原子性和持久性，必须使执行事务T的所有节点在T执行的最终结果上取得一致 -- 在所有节点上都提交，或在所有节点上都中止。
- 由事务T的事务协调器执行一个提交协议来保证这一点。
  - ✓ 两阶段提交协议(2PC)
  - ✓ 三阶段提交协议(3PC)

10.6 分布式事务处理

➢两阶段提交协议

- 在分布式事务处理过程中，某些场地可能发生故障，或者由于网络发生故障而无法访问到某些场地。
- 为了防止分布式系统部分场地发生故障产生数据的不一致性，在分布式事务管理中采用了两阶段提交协议(Two-Phase Commit Protocol)。

10.6 分布式事务处理

➢两阶段提交协议

- 提交协议:当执行T的所有节点(参与者)都通知事务协调器Ci已完成了T的执行时，Ci启动2PC协议。
- 阶段1:投票阶段
  - ✓Ci将记录<prepare T>加到日志中，并强制日志写入稳定存储器中。接着将<prepare T>消息发送到执行T的所有节点。
  - ✓当参与节点收到<prepare T>消息时，节点上的事务管理器确定是否提交T中属于它的部分。
  - ✓若答案是“不”，就把记录<no T>加到日志中，并向Ci发送一条<abort T>消息来作出回答。
  - ✓若答案是“是”，就把记录<ready T>加到日志中，并将日志(包括所有与T相关的日志项)强制写入稳定存储器中，然后向Ci发送一条<ready T>消息来作出回答。

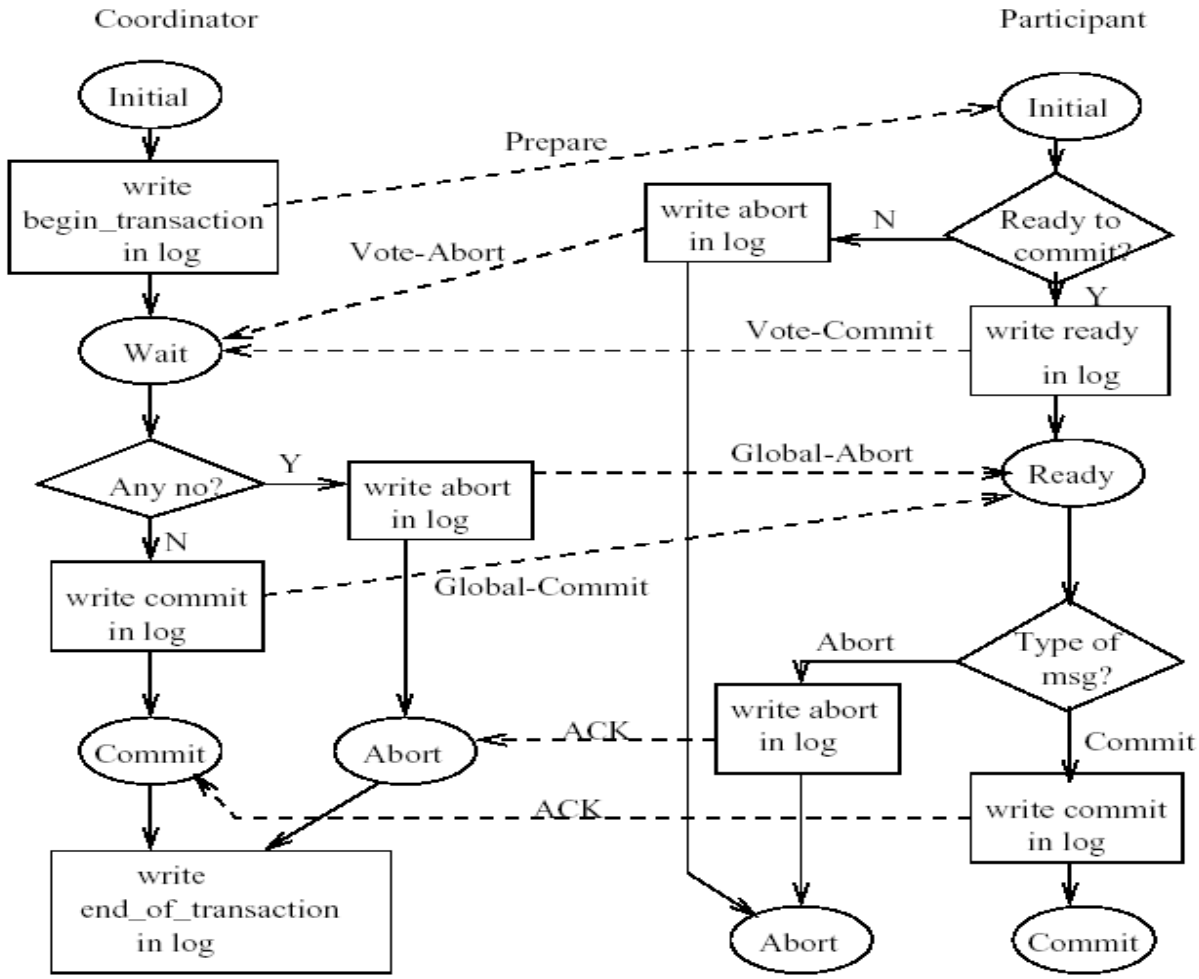
10.6 分布式事务处理

➢两阶段提交协议

- 阶段2:决定阶段
  - ✓当C_i收到所有节点对<prepare T>消息的回答时，或者从<prepare T>消息发送后一个预定的时间间隔已经过去时，Ci就可以确定是将事务T提交还是中止。
  - ✓如果C_i从所有参与的节点都收到<ready T>消息，事务T就可以被提交。
  - ✓否则，事务T必须中止。
  - ✓根据结论的不同，将记录<commit T>或记录<abort T>加到日志中，并将日志强制写入稳定的存储器中。
  - ✓然后，协调器向所有的参与节点发送消息<commit T>或消息<abort T>
  - ✓当参与节点上收到此消息，就把消息记录到日志中。

10.6 分布式事务处理

➢两阶段提交协议



10.6 分布式事务处理

➢分布式并发控制

- 对集中式系统的并发控制模式进行修改，以适用于分布式环境，需考虑：
  - ✓封锁管理器的位置
  - ✓多副本的封锁
  - ✓全局的死锁处理
- 等问题

10.6 分布式事务处理

➢分布式并发控制

- 方法:
  - ✓单一锁管理器方式
  - ✓多协调器方式
  - ✓多数协议
  - ✓有偏协议
  - ✓主副本
  - ✓死锁处理

10.6 分布式事务处理

➢分布式并发控制

- 单一锁管理器方式
  - ✓整个系统维护一个锁管理器，位于选定的节点Si上。所有封锁和解锁请求都在节点Si处理。
- 优点：
  - ✓实现简单
  - ✓死锁处理简单
- 缺点：
  - ✓节点Si成为瓶颈
  - ✓脆弱性

10.6 分布式事务处理

➢分布式并发控制

- 多协调器方式
  - ✓系统维护位于几个节点上的几个封锁管理器，每个封锁管理器管理封锁和解锁请求的一个子集。
  - ✓优点：降低了协调器成为瓶颈的程度。
  - ✓缺点：死锁处理变得复杂。

10.6 分布式事务处理

➢分布式并发控制

- 多数协议
  - ✓以一种分散的方式处理复制数据，每个节点维护自己局部的锁管理器，管理对存储在该节点上的数据所进行的封锁和解锁请求。
  - ✓当事务希望封锁存储在节点S_i且未被复制的数据项Q时，发送请求封锁的消息给节点S_i上的锁管理器。
  - ✓如果数据项Q在n个不同的节点上被复制，则封锁请求必须送到存储Q的n个节点的一半以上节点上去。
  - ✓只有获得Q的多数副本上的封锁，事务才开始Q上的操作。

10.6 分布式事务处理

➢分布式并发控制

- 优点：
  - ✓ 当数据只有一个副本时实现简单
- 缺点：
  - ✓当存在多个数据副本时实现复杂
  - ✓死锁处理复杂

10.6 分布式事务处理

➢封锁协议

- 有偏协议
  - ✓类似于多数协议，但共享锁请求受到的待遇比排他锁请求受到的待遇要优越一些。
  - ✓每个节点上维护一个锁管理器，管理存储在该节点的所有数据项上的锁。
  - ✓ 共享锁请求：事务需要封锁数据项Q时，只需要向包含Q的副本的一个节点上的锁管理器请求封锁。
  - ✓排他锁请求：事务需要封锁数据项Q时，需要向包含Q的副本的所有节点上的锁管理器请求封锁。

10.6 分布式事务处理

➢封锁协议

- 优点：
  - ✓read操作上的开销比多数协议小
- 缺点：
  - ✓write操作的开销比多数协议大
  - ✓死锁处理复杂

10.6 分布式事务处理

➢封锁协议

- 主副本
  - ✓选择数据的多个副本中的一个作为主副本。数据项Q的主副本所位于的节点称为Q的主节点。
  - ✓事务需要封锁数据项Q时，在Q的主节点上请求封锁。
  - ✓优点：实现简单
  - ✓缺点：可用性降低

10.6 分布式事务处理

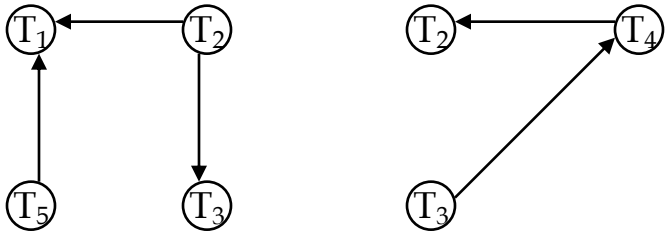
➢封锁协议

- 死锁处理
- 通过等待图进行死锁检测的方法
  - ✓局部等待图：每个节点维护一个局部等待图，图中节点对应于目前占有或请求该节点上任何数据项的事务，边$T_i \rightarrow T_j$表示事务T_i等待事务T_j释放所需数据项。
  - ✓全局等待图：各个节点的局部等待图的并。

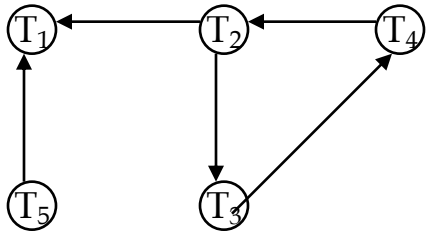
10.6 分布式事务处理

➢封锁协议

- 如果某个局部等待图中存在环，则发生了死锁。
- 如果任何局部等待图中都不存在环，也可能发生了死锁，因为可能全局等待图中存在环。



局部等待图



全局等待图

10.6 分布式事务处理

➢封锁协议

- 组织等待图的方式：

  ✓集中方式：由单一的一个节点 — 死锁检测协调器构造和维护全局等待图。

  ✓全分布方式：所有的节点平等地承担检测死锁的责任。每个节点根据系统的动态行为构造一个代表整个图的一部分的等待图。其思想是：如果死锁存在，则至少一个部分图中会存在环。

10.7 联邦数据库和多数据库系统

➢联邦数据库

- 定义：

  联邦数据库系统是松耦合的分布式数据库系统，无论在物理上还是在逻辑上都是分布的，各结点可以采用不同的DBMS。

- 特点：

  ✓无全局数据模式，只有各的结点局部数据模式。

  ✓结点间的数据共享由双边协商确定。

  ✓有利于实现数据库的集成、扩展和重新配置。

10.7 联邦数据库和多数据库系统

➢联邦数据库系统的功能

- 实现联邦系统中的数据共享和软、硬件资源的合并。
- 实现数据的透明访问。
- 提供高效的数据访问机制和协调访问的并发控制机制。
- 保证各局部数据库系统的自治能力。
- 提供多种数据模型和数据语言以及它们间的转换能力，以保证用户方便地共享数据。

10.7 联邦数据库和多数据库系统

➢联邦数据库的数据共享

- 联邦数据库系统在实现数据共享时能使用户以在同构数据库中一样的方式访问异构数据库，用户可以用自己熟悉的局部数据库语言及模型访问异构数据库。
- 允许数据库的拥有者和其他用户共享数据库，而又不放弃自己数据库的完整性限制、应用特性和安全性限制，保证异构数据库的局部自治能力。

10.7 联邦数据库和多数据库系统

➢联邦数据库的数据共享

- 系统是多模型、多语言的，即可采用多种数据模型描述相应的应用数据，可执行以多种语言写的事务。
- 为了保证各局部数据库的自治，维护各局部数据库的完整性、安全性要求以及应用特性，对局部数据库的访问必须加以控制。

10.7 联邦数据库和多数据库系统

➢联邦数据库的数据共享

- 联邦数据库系统必须提供访问控制和并发机制，以保证系统中各片段数据的完整性限制不被破坏，并同时提供特殊的数据模型和数据语言的接口，加强必要的安全性限制，以协调联邦数据库中各局部数据库的并行访问。
- 高效正确的数据访问机制和并发控制机制是保证局部数据库自治的必要条件，是联邦数据库真正有效实现数据共享和高效率的资源共享要求的保证。
- 协调数据访问及并发机制的能力是衡量联邦数据库系统性能的一个重要指标。

10.7 联邦数据库和多数据库系统

➢联邦数据库的数据共享

- 联邦数据库系统中实现数据共享的方法有：
  - ✓数据库转换方法
  - ✓模式转换方法

10.7 联邦数据库和多数据库系统

➢联邦数据库的数据共享

- (1) 数据库转换
  - ✓将数据库转换成用户熟悉的等价数据模型，然后再装入数据，使用户方便地使用新的数据库以实现共享。
  - ✓这是一种物理方法，实质上包括了模型转换和数据重组两项工作，由数据库转换器完成。

10.7 联邦数据库和多数据库系统

➢联邦数据库的数据共享

- 数据库转换具体转换过程：
  - ✓首先访问源数据库系统，拷贝一个源数据库的副本；
  - ✓然后将源数据库的副本转换成等价的目标数据库，并装入到与目标数据库对应的数据库系统；
  - ✓最后用户以目标数据库管理系统中的数据处理语言访问目标数据库。
- 利用数据库转换器实现的数据共享体现了异构数据库的透明访问。

10.7 联邦数据库和多数据库系统

➢联邦数据库的数据共享

- (2) 模式转换
  - ✓通过模型的转换、数据语言翻译、建立转换映射以及转换操作数据的事务来实现数据共享，是一种逻辑方法
  - ✓步骤：
  - ✓首先将异构数据库的数据模型转换成用户熟悉的数据模型，用户使用熟悉的数据语言编写事务对数据进行处理，处理的执行可通过事务翻译器转换为对异构数据的操作后进行
  - ✓模式转换只涉及模型转换和事务翻译，不产生数据的副本

10.7 联邦数据库和多数据库系统

➢联邦数据库的数据共享

- 在联邦数据库系统中为了达到较高程度的异构，必须支持多种异构的数据库系统在不同的计算机上运行。
- 除多对一映射方式外，目前的技术还不足以将很多的不同计算机联接在一起以进行数据共享，目前系统的异构程度还相当低。

10.7 联邦数据库和多数据库系统

➢多数据库系统

- 多数据库系统(MDBS)是一种松耦合的异构分布式数据库系统。
- 从软件结构角度看，多数据库系统可看成是加在系统的各数据库管理系统之上的一层软件，它使用户可从任一场地对系统中其他场地的数据库进行存取。

10.7 联邦数据库和多数据库系统

➢多数据库系统

- 实现MDBS的技术有：
  - ✓(1) 远程数据库存取，即RPC，已经有许多成熟技术；
  - ✓(2) 多数据库系统的并发存取控制和事务管理

10.7 联邦数据库和多数据库系统

➢多数据库系统

- MDBS中全局事务由若干子事务组成，每个子事务只存取某一个LDBS。
- MDBS可看成由全局事务管理系统(GTMS)加上参加该MDBS的各LDBS顶上的服务器(LS)组成。
- LDBMS自治地调度其上的子事务，在此基础上GTMS在各LS支持下保证整个MDBS中数据的一致性。

10.7 联邦数据库和多数据库系统

➢多数据库系统

- MDBS中的事务语义协议
- 目标：利用事务的语义和放宽‘可串行’的准则，区分：
  - ✓可局部化事务：子事务在局部数据库管理系统执行的顺序不影响全局事务的正确性。
  - ✓不可局部化事务：子事务在局部数据库管理系统执行的顺序影响全局事务的正确性。
- MDBS并发控制方法保证并发执行的非可局部化的全局事务从等价一致的MDBS中读，同时保证所有全局事务的原子性。

10.7 联邦数据库和多数据库系统

➢多数据库系统

- 目标：利用事务的语义和放宽 ‘可串行’ 的准则，区分：
  - ✓MDBS中的事务语义协议：
  - ✓GTMS通过LS向LDBS分发全局事务的子事务，并指明其类别。
  - ✓LDBS向LS报告子事务的夭折、提交或所有运算已完成，LS等待2PC过程决定提交后才向LDBS发出提交指令。
  - ✓LDBS完全自治。
  - ✓可以证明，MDBS中遵守“事务语义协议”的任何调度都是正确的。

10.7 联邦数据库和多数据库系统

➢多数据库系统

- MDBS调度算法：
- A．收到的子事务是局部子事务

If 其时间戳小于正运行或已提交的非局部子事务的最大时间戳 then

拒绝接受，投天折票；

else if 无非局部子事务在运行且无时间戳更小的非局部子事务在等待 then

将该子事务交给LDBS执行，投提交票；

else 子事务进入等待表，投提交票

10.7 联邦数据库和多数据库系统

➢多数据库系统

- MDBS调度算法
- B．收到的子事务是非局部子事务

If 其时间戳小于正运行或已提交的子事务的最大时间戳 then

拒绝接受，投天折票；

else if 该LDBS空闲且无时间戳比该子事务的时间戳更小的子事务在等待这个LDBS then

将该子事务交给LDBS执行；

Else 子事务进入等待表

- C．当子事务提交后LDBS空闲

将时间戳最小的等待子事务交给LDBS执行；

If 这是局部子事务then

将等待表中时间戳小于表中所有非局部子事务的时间戳的局部子事务都交LDBS执行

10.7 联邦数据库和多数据库系统

➢多数据库系统

- 全局事务提交的方法
  - ✓在2PC中，GTMS是协调者，分发子事务时要求各LDBS的LS代表其上执行的子事务参加投票。
  - ✓对非局部子事务，LS收到LDBS的夭折消息则投夭折票。
  - ✓收到完成报告则投提交票，并进入等待状态，直到收到协调者的提交命令时，才向LDBS发出提交命令，子事务真正提交。

10.7 联邦数据库和多数据库系统

➢多数据库系统

- 全局事务提交的方法
  - ✓如LS投提交票后，LDBS由于某种原因(如打破死锁)而夭折该子事务，若协调者决定夭折这个事务，则不用做任何事。
  - ✓若协调者决定提交，则LS将重新递交夭折了的子事务给LDBS，称为重做子事务，如再次夭折了，将再次重做，直到成功地提交为止。

10.7 联邦数据库和多数据库系统

➢多数据库系统

- 全局事务提交的方法
  - ✓对局部子事务则稍有不同，LS在收到子事务时就投票，让协调者尽快知道是否会夭折。
  - ✓以后LS要等到从协调者收到了提交命令，又接到LDBS的完成报告后才向LDBS发提交命令。
  - ✓如LS从协调者收到了提交命令而LDBS报告夭折，LS也将重新递交这个子事务给LDBS，直到完成为止。
  - ✓任何时候如果 LS从协调者收到夭折命令而子事务还在执行，则向LDBS发夭折命令。

10.8 并行数据库系统

➢并行数据库系统

- 什么是并行数据库系统?
- 并行数据库的目标
- 并行计算机系统的体系结构
- 并行粒度
- 并行方式
- 数据划分

10.8 并行数据库系统

➢什么是并行数据库系统?

- 并行数据库系统是在硬件多处理器并行机或集群系统上运行的具有并行处理能力的高性能和高可用性数据库系统，是数据库技术与并行计算技术结合的产物。
- 并行数据库系统是新一代高性能的数据库系统，致力于开发数据库操作的时间并行性和空间并行性，是当今研究热点之一。
- 并行数据库系统设计面临的问题：并行I/O、并行查询优化、并行性数据库操作等。

10.8 并行数据库系统

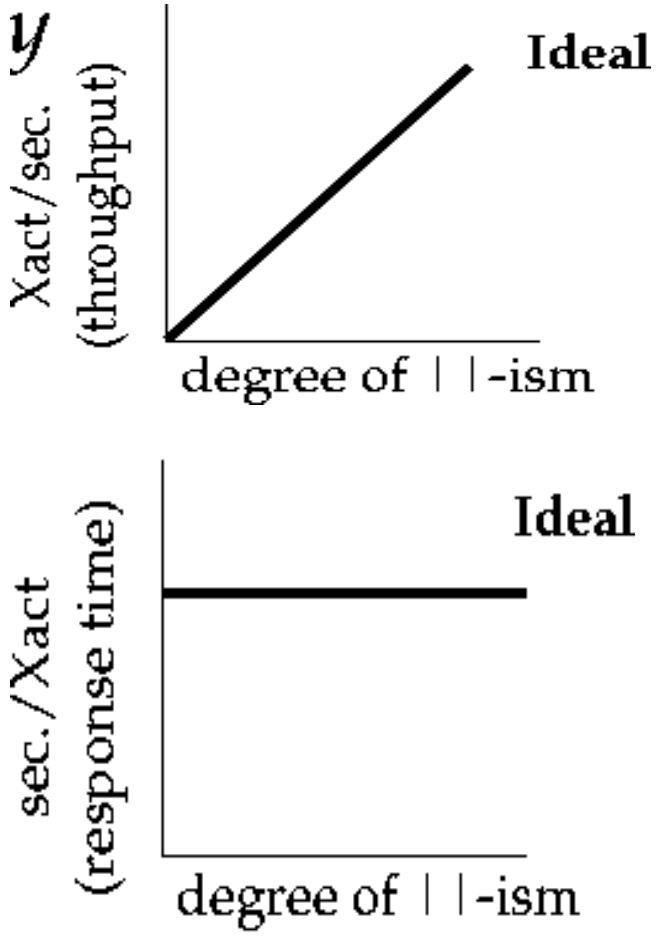
➢并行数据库的目标

- 1)高性能：通过将数据库在多个磁盘上分布存储，利用多个处理机对磁盘数据进行并行处理，解决I/O瓶颈问题。通过开发查询间并行性、查询内并行性以及操作内并行性，提高查询效率。
- 2)高可用性：可通过数据复制来增强数据库的可用性，当一个磁盘损坏时，该盘上的数据在其他磁盘上的副本仍可供使用。
- 3)可扩充性：系统通过增加处理和存储能力而平滑地扩展性能的能力。

10.8 并行数据库系统

➢并行数据库的目标

- 衡量可扩充性的主要指标包括：
  - 加速比(speedup)：对于某个固定的计算任务，1倍计算资源系统所完成的时间与n倍计算资源所完成时间之比；理想的speedup曲线为线性加速。
  - ✓伸缩比(scaleup)：将1倍计算任务在1倍计算资源系统所完成的时间与n倍计算任务在n倍计算资源系统所完成时间之比，理想的scaleup曲线为y=1。



10.8 并行数据库系统

➢影响并行数据库系统性能的三个因素

- 启动代价(startup)：启动一个并行操作的代价。
- 干扰(interference)：共享资源之间的相互干扰。
- 倾斜(skew)：一个操作或一个查询可以被分解为若干个可并行执行的子操作或子查询，执行时间最长的子操作或子查询的响应时间决定该并行操作的响应时间。

10.8 并行数据库系统

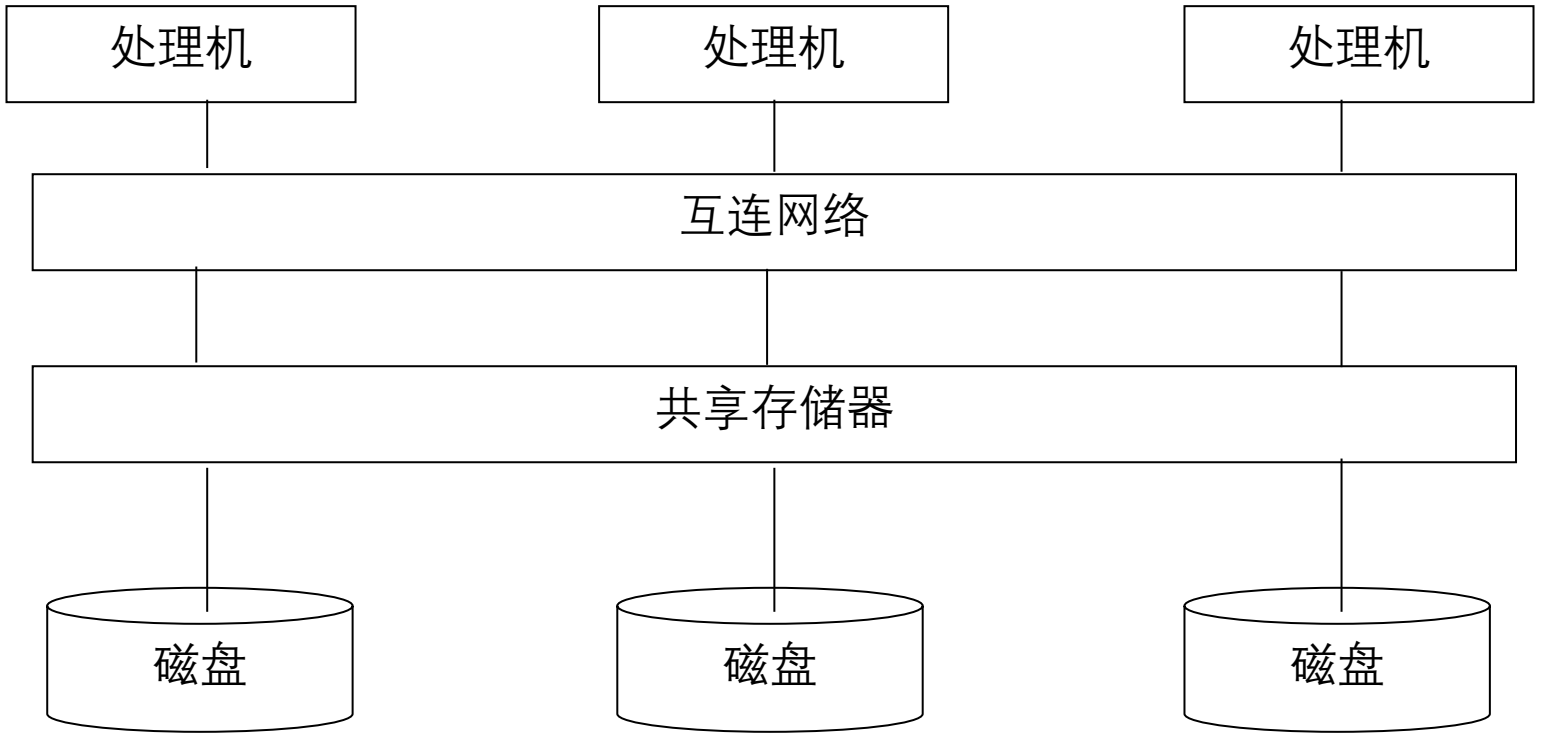
➢数据库中的并行性

- 数据可划分在多个磁盘上, 导致I/O并行。
- 个别关系操作 (如排序, 连接, 合计) 可以并行执行
  - ✓数据可划分且每个处理器可对其划分部分独立操作。
- 查询可用高级语言(SQL, 翻译到关系代数)表达
  - ✓使并行化更容易。
- 不同查询可并行执行
  - ✓ 并发控制处理冲突。
- 数据库很自然地具有并行性。

10.8 并行数据库系统

➢并行计算机的体系结构

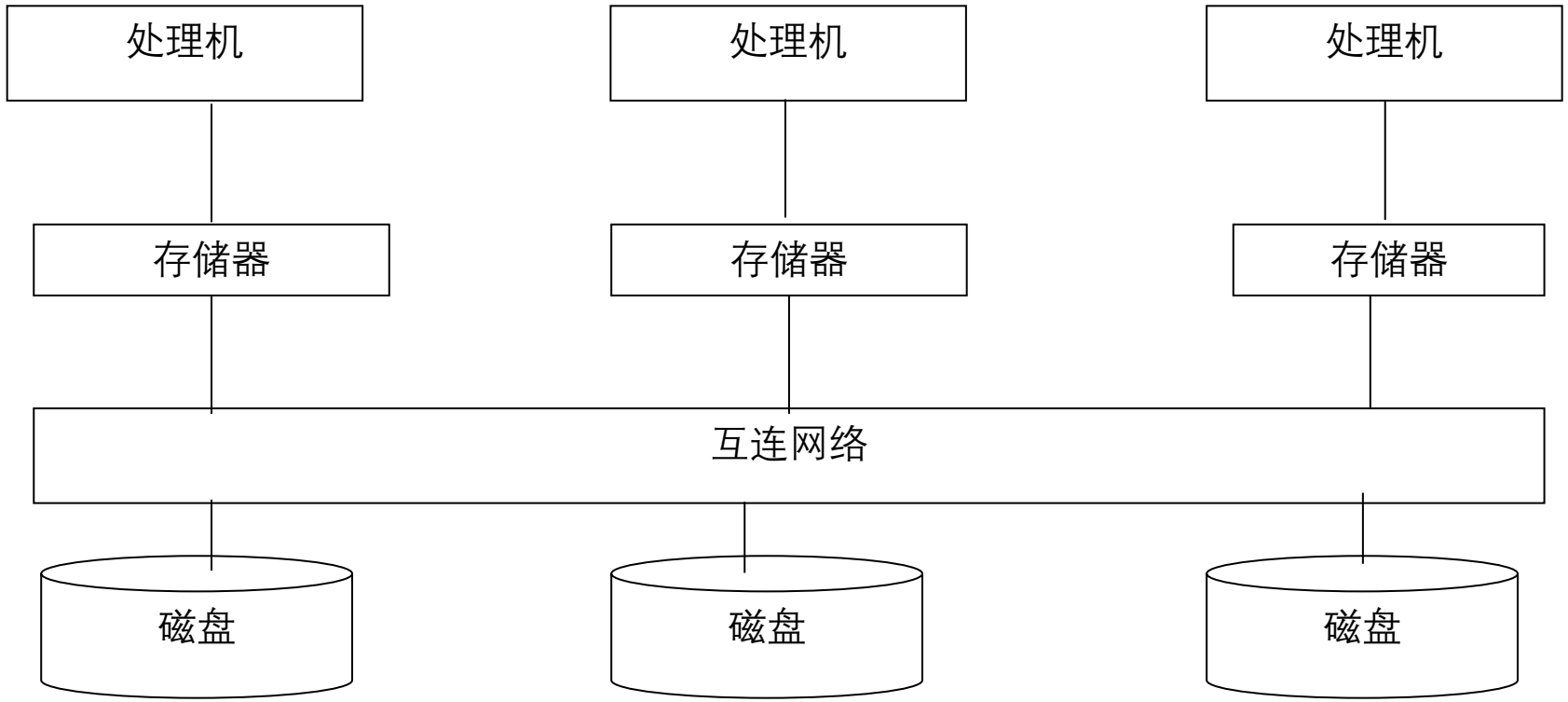
- 共享内存(Shared-Memory，SM)结构：



10.8 并行数据库系统

➢并行计算机的体系结构

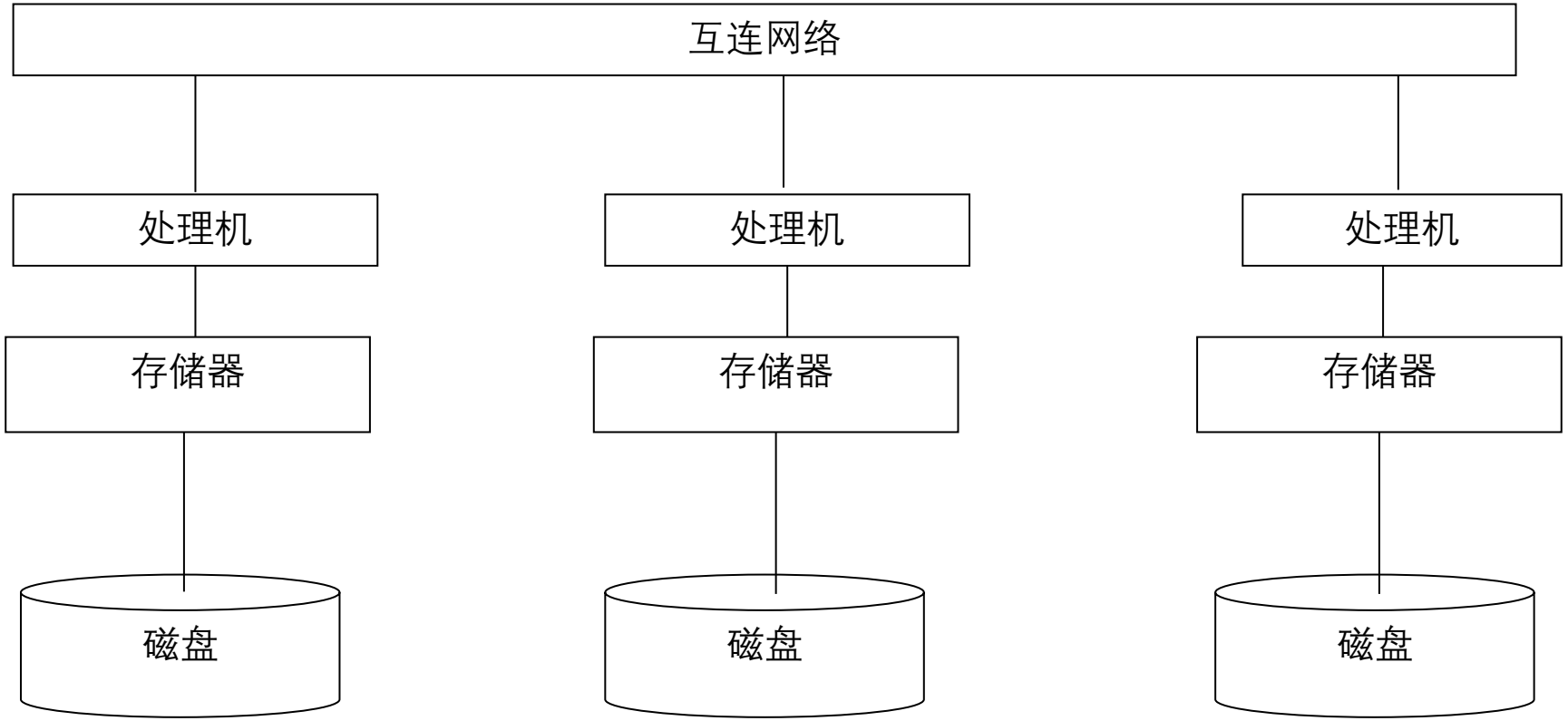
- 共享磁盘(Shared-Disk，SD)结构：



10.8 并行数据库系统

➢并行计算机的体系结构

- 无共享资源(Shard-Nothing，SN)结构：



10.8 并行数据库系统

➢并行计算机的体系结构

- 三种并行结构比较

	共享内存	共享磁盘	无共享资源
性能	最佳	较佳	较佳
可用性	低	较高	高
可扩充性	差	较好	好
负载均衡	易做到	易做到	难做到
实现技术	容易	较复杂	复杂
成本	高	较低	低
处理机数	数十个	数百个	数千个
规模	中小系统	中小系统	大系统

10.8 并行数据库系统

➢并行粒度

- 事务间(Inter-Transation)并行：是粒度最粗也是最容易实现的并行性由于这种并行性允许多个进程或线索同时处理多个用户请求，因此可以显著增加系统吞吐量，支持更多的并发用户。
- 查询间(Inter-Query)并行性：也称为事务内(Intra Transation)并行，同一事务内的不同查询如果是不相关的，它们并行执行必将提高效率，但是，同一事务内的查询如果是相关的，它们并行执行比较复杂，系统必须进行相关性控制。

10.8 并行数据库系统

➢并行粒度

- 操作间(Inter-Operation)并行性：也称为查询内(Intra-Query)并行性，同一查询内的不同操作往往可以并行执行。考虑一条SQL查询语句可以分解成多个子操作，有多个处理机执行。
- 操作内(intra-Operation)并行性：是粒度最细的并行性，它将同一操作(如扫描操作、合并操作、排序操作等)分解成多个独立的子操作，由不同的处理机同时执行。

10.8 并行数据库系统

➢并行粒度

事务(Transation)			事务内	事务间
查询(Query)		查询内	查询间	
操作(Operation)	操作内	操作间		

并行粒度　　细 ——————————→ 粗

图　4种并行粒度

10.8 并行数据库系统

➢并行方式

- 水平并行化(独立并行化，Independent Parallelism)：将互相独立的多个操作或者一个操作内互相独立的多个子操作分配给不同的处理机并行执行。
- 垂直并行化(流水线并行化，Pipelining Parallelism)：将存在流水线方式依赖关系的多个操作分配给不同处理机并行执行。

10.8 并行数据库系统

➢并行方式

- 例：对于下列查询：

```
SELECT DEPTNUM , EMPNUM
    FROM DEPT D, EMP E
        WHERE D.DEPTNUM = E.DEPTNUM
            GROUP BY DEPTNUM
                ORDER BY DEPTNUM;
```

- 可以分解为扫描DEPT(部门)表和EMP(职工)表、对两表进行连接、对连接结果排序以及分组和输出五个子任务。
- 前一子任务的输出即是下一子任务的输入，后一子任务等待前一子任务产生一定量的输出后(而不必等待前一子任务执行完毕)即可在另一处理器上开始执行。
- 这种并行方式称为垂直并行或流水线并行。

10.8 并行数据库系统

➢数据划分

- 从广义上讲，事务间并行和查询间并行都属于水平并行。
- 水平并行性要求物理地将数据库划分为较小分片，并存放在不同磁盘上，这就是并行数据库系统中的数据划分。

10.8 并行数据库系统

➢数据划分

- round-robin划分:按顺序将数据依次分布到多个磁盘上。
- range划分:按某个属性的值域来划分数据。
- hash划分:使用一个经过高度调优的系统函数对划分码进行Hash，并据此确定相应行的分片。
- user-defind划分:是指由用户定义对表进行划分。它使数据的划分更为灵活，可以满足不同用户的特殊应用需要。
- schema划分:是指不对表进行划分，而以表为基本单位分布数据。

课后作业

➢**P243**

- 10.2
- 10.7
- 10.12